# Binary Linear System을 위한 일반화 가능한 학습된 branching

Generalisable Learned Branching for Binary Linear Systems
Gwang-Jong Ko, Taesu Cheong*, In-Chan Choi* · School of Industrial and Management Engineering, Korea University · 2026-10-12 · 한국어판

## 초록

Binary linear system(BLS)은 0/1 행렬 A에 대해 $A\mathbf{x}=\mathbf{b}$를 만족하는 vector $\mathbf{x}\in\{0,1\}^n$를 찾는 문제다; 우리는 해가 존재하고 그것을 내놓아야 하는 해 찾기 버전을 다룬다. 목적함수가 없고 linear relaxation은 넓고 무정보하며 unit propagation은 root 근처에서 작동하지 않으므로, complete depth-first search의 성패는 branching 선택이 결정한다. 학습된 branching은 mixed-integer program과 SAT를 위해 개발됐고, 우리는 그 세 주요 계열이 BLS로 옮겨오면 각각 무언가를 잃음을 보인다: strong branching 모방은 개선할 bound가 없어 LP probing으로 퇴화하고, message-passing marginal은 밀집 factor graph에서 node당 network forward pass의 두 자릿수 배 비용이 들며, from-scratch 강화학습은 문제가 정확히 제공하는 값 순서를 스스로 발견해야 한다. 본 논문은 두 개의 정확한 신호로 두 단계에 걸쳐 학습한 branching 규칙을 제시한다. **Stage 1**은 **어느 값**을 먼저 시도할지 배운다: 열거 가능한 instance에서 각 자유 변수의 conditional marginal은 정확하고, BLS를 partial assignment로 conditioning하는 것은 축소하는 것과 같으므로 축소 instance로 학습한 graph network 하나가 모든 node에서 예측한다. **Stage 2**는 **어느 변수**로 분기할지 배운다: 결정의 subtree가 전개하는 node 수는 탐색이 정확히 산출하며, Stage 1 규칙에서 출발해 network를 동결한 채 그 비용으로 actor-critic fine-tuning하면 branching 순서만 바뀐다. 고전 규칙 7종(most-integral·most-fractional LP branching, maxSD counting, dom/wdeg, impact, 전수 LP probing, uniform)과 참조 신호 2종(정확한 marginal, belief propagation)을 같은 탐색 루프에서, 세 크기 100개 세트로 평가한다. 학습 규칙은 모든 크기에서 단일 relaxation 규칙 전부보다 node를 적게 쓰고(가장 강한 규칙 대비 1.5~3.2배, sign test $p\le0.035$), $28\times80$에서는 시간도 전부보다 빠르다(2.0~3.1배, $p\le0.002$); LP probing만이 node를 더 적게 쓰되 5~8배의 시간이 든다. $n\le60$에서 학습한 규칙은 $n=70, 80$에서 우위를 유지하고 그 우위는 크기와 함께 커진다. 값×변수 교차 ablation은 이득이 변수 선택에 있음을 보인다: 비용 조정 변수에 LP 반올림 값을 붙이면 전체 규칙과 같고, LP 변수에 정확한 marginal 값을 붙이면 28배 나쁘다. SCIP에 branching 규칙으로 꽂으면 policy는 conflict analysis on/off 모두에서 reliability pseudocost branching보다 node를 1.9~3.8배 적게 쓰고 큰 두 크기에서 시간은 동률이며, presolve와 clause learning을 갖춘 complete solver는 end-to-end에서 여전히 더 빠르다; 격차는 tree 크기가 아니라 node당 비용에 있다.

**키워드**: Binary linear systems · Learning to branch · Exact conditional marginals · Reinforcement learning · Depth-first search · Graph neural networks

## 1. 서론

Binary linear system(BLS)은 해를 찾는 문제

$$\text{find } \mathbf{x}\in\{0,1\}^n \text{ such that } A\mathbf{x}=\mathbf{b},\ A\in\{0,1\}^{m\times n},\ \mathbf{b}\in\mathbb{Z}^m_{\ge0} \quad (1)$$

이며 해집합을 S로 쓴다. 레이저 기반 비파괴검사에서 이진 점유 영상이 정수 투영 집합을 정확히 재현해야 하는 복원 과제의 일반형이다: 스캔 대상은 존재하므로 해가 존재하고 유일하지 않은 경우가 흔하며, 복원은 생산 라인의 속도를 따라가야 한다. 본 논문 전체에서 instance에는 항상 해가 있고 과제는 그 해를 내놓는 것이다; 시스템이 infeasible한지 판정하는 것은 범위 밖이다(3.1절). 중요한 instance는 우변이 각 행 범위의 한가운데 근처에 있으며, 크기에 비해 어렵다.

이런 instance에서는 세 가지 성질이 표준 도구를 무력하게 만든다. 목적함수가 없으므로 branch-and-bound의 bound 기반 가지치기가 기댈 것이 없다. linear relaxation은 항상 feasible하지만 정보가 적다: relaxation polytope가 넓고 그 vertex가 $\{0,1\}^n$에서 멀어, vertex를 반올림해도 root에서 변수 값을 우연보다 조금 나은 정도로만 맞힌다. constraint solver의 주력인 unit propagation은 행이 포화될 때만 발동하는데 root에서는 결코, 그 근처에서도 거의 일어나지 않는다. 이 계열의 고전적 처방인 enumeration을 갖춘 lattice basis reduction은 건전하고 우리가 다루는 모든 instance를 풀지만 비용이 complete search보다 크기에 따라 빠르게 커진다; lattice가 남기는 instance와 그것이 멈추는 크기에서 complete search를 결정하는 것은 branching 결정의 품질이고, 그것이 본

논문의 대상이다.

### 1.1 학습된 branching이 여기에 남겨 둔 것

세 계열의 연구가 branching 결정을 학습하며 2절이 검토한다: strong branching 모방, message passing에 의한 근사 marginal, subtree 크기에 대한 강화학습. 각각은 목적함수가 있는 문제, 희소한 제약 graph, 또는 희소한 보상을 위해 개발됐고, BLS에 적용하면 각각 무언가를 잃는다. 목적함수가 없으면 strong branching은 점수 매길 bound 개선이 없어 node마다 자유 변수당 relaxation 하나를 지불하는 규칙인 LP probing으로 퇴화한다. BLS의 밀집 factor graph에서 belief propagation은 강한 damping과 추정 한 번에 수백 번의 반복이 필요하고, root 수준의 사용법은 complete search가 모든 node에서 필요로 하는 conditional marginal을 제공하지 않는다. from-scratch 강화학습은 어느 변수로 분기할지를 가르치는 같은 희박한 신호로부터 어느 값을 먼저 시도할지를 스스로 발견해야 하지만, BLS는 열거할 수 있을 만큼 작은 instance에서 최적의 첫 값을 정확히 제공한다. 빠진 것은 BLS가 제공하는 정확한 신호 — 자유 변수의 conditional marginal과 branching 결정의 정확한 비용 — 를 취해 값과 변수를 따로 학습하고, 그 신호를 감당할 수 있는 크기에서 한 번 학습해 감당할 수 없는 크기에 배포하는 branching 규칙이다. 본 논문은 그런 프레임워크를 제시하고(4절) 고전 규칙과 세 전이형에 대해 같은 탐색 루프에서 평가한다.

### 1.2 연구 질문

실험은 네 질문으로 구성되며, 각각 세 크기 100개 세트에서 모든 branching 규칙을 같은 탐색 루프로 돌려 답한다. **RQ1** 학습 규칙이 고전 branching 규칙과 전이된 learned-branching baseline 3종을 node와 시간 **모두**에서 이기는가? 작은 tree가 시간을 사는 것은 node당 비용이 허락할 때뿐이므로 그 비용이 어디로 가는지도 포함한다. **RQ2** $n \leq 60$에서 학습한 규칙이 더 큰 instance로 일반화되는가, 그 우위는 크기에 따라 줄어드는가 커지는가? **RQ3** 두 신호는 서로 대체 가능한가, 각 단계는 필요한가? 비용 조정 policy가 탐색에서 실제로 무엇을 바꿨는지를 포함한다. **RQ4** complete solver 안의 branching 규칙으로서, 그리고 end-to-end로 complete solver와 겨루면 어떠한가?

### 1.3 기여

(1) branching 결정의 두 부분을 문제의 두 정확한 신호로 학습하는 BLS용 학습 branching 프레임워크 — 값은 열거 가능한 곳에서 학습해 reduction 항등식으로 complete search의 모든 node로 옮기고, branching 순서는 값 모델을 동결한 채 정확한 subtree 비용으로 학습하므로 이득이 귀속되고 모델이 본 적 없는 크기로 전이된다. (2) RQ1~RQ4에 대한 통제된 답: 고전 규칙 7종·참조 신호 2종·전이된 학습 baseline 3종을 한 루프에서, 세 크기 100개 세트, 값×변수 교차 ablation, 학습 변형 ablation, 비용 신호가 가르친 것의 반증 구조 분석, SCIP branching plug-in으로서 reliability pseudocost branching과의 비교, SCIP·CP-SAT·Gurobi·lattice enumeration과의 end-to-end 비교. (3) 그렇게 보고하는 두 부정적 발견: 변수 선택을 학습하고 나면 정확한 marginal 값은 LP 반올림보다 나을 것이 없고, 학습 규칙은 complete solver와의 wall-clock 격차를 닫지 못한다.

2절은 관련 연구를, 3절은 문제와 reduction 항등식을, 4절은 방법을, 5절은 연구 질문별 실험을, 6절은 한계와 함께 결론을 다룬다.

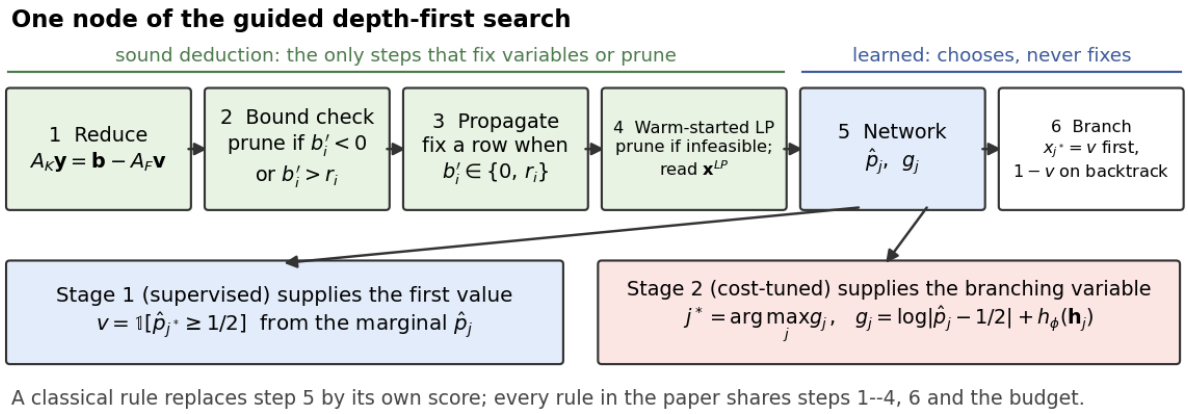



그림 1. guided depth-first search. node 안에서 1~4단계는 건전하며 변수를 확정하거나 가지를 치는 유일한 단계다; 5단계의 network는 선택만 하며, 값은 Stage 1에서, 변수는 Stage 2에서 온다. lattice reduction은 쓸 때 탐색 앞의 전처리로 돌며 모든 branching 비교에서는 끈다.

## 2. 선행연구

### 2.1 해집합의 marginal을 branching 신호로

결정에 동의하는 해의 비율로 분기하는 것은 학습 이전부터 있었다: counting-based search(Pesant et al., 2012)는 개별 제약 안에서 계산한 solution density가 최대인 변수-값으로 분기하고, belief-propagation-guided decimation(Montanari et al., 2007)은 근사 marginal로 가장 편향된 변수를 고정한다. NSNet(Li and Si, 2022)은 SAT 공식의 만족 배정을 열거해 얻은 정확한 marginal로 graph network를 학습하고 추정치를 반올림해 local search의 초기 배정으로 쓴다; MIP-GNN(Khalil et al., 2022), Neural Diving(Nair et al., 2020), predict-and-search(Han et al., 2023)는 수집한 준최적 해로부터 변수별 bias를 예측해 변수 고정·warm start·탐색 제한에 쓴다. 이들 모두에서 marginal이나 bias는 root에서 한 번 추정되어 변수를 고정하거나 incomplete search를 시작하는 데 쓰인다; complete search의 모든 node에서 partial assignment에 conditional인 marginal — branching 규칙이 소비하는 것 — 을 제공하는 것은 없다.

### 2.2 Learning to branch

Khalil et al.(2016)과 Gasse et al.(2019)은 strong branching을 모방하며, 후자는 우리도 쓰는 bipartite 변수–제약 graph network를 쓴다; 모방은 순위가 줄이려는 tree 크기가 아니라 expert의 순위를 target으로 삼는다. tree 크기를 직접 최적화하는 것은 강화학습 연구의 한 줄기다: FMSTS(Etheve et al., 2020)는 node의 subtree 크기를 관측 가능한 Q-value로 쓰고 depth-first search에서는 모든 subtree를 최소화하면 전체 tree가 최소화됨을 보이며 from scratch로 학습한다; tree MDP(Scavuzzo et al., 2022)는 subtree 단위 credit assignment를 형식화하고, retro branching(Parsonson et al., 2023)은 회고적으로 추출한 subtree 경로에서 학습하며, SORREL(Feng and Yang, 2025)은 시연으로 초기화한다. Neuro#(Vaezipoor et al., 2021)은 #SAT solver의 residual formula 위의 graph network를 evolution strategies로 학습해 더 큰 instance로 일반화한다. 강화학습을 정확한 solver 안에서 쓰는 더 넓은 흐름은 Mazyavkina et al.(2021)이 정리했고, operations research 문헌에서도 학습된 policy·value network를 heuristic tree search 안에 둔 사례(Hottung et al., 2020)와 제약 구조 위의 graph network가 그런 학습 heuristic의 표준 표현이 된 흐름(Smit et al., 2025)이 있다. 이 방법들이 최적화하는 목적의 고전적 진술이 fail-first 원칙(Haralick and Elliott, 1980)과 impact-based search(Refalo, 2004)다; bound가 아니라 feasibility가 어려운 mixed-integer program에서는 relaxation의 변화를 강제하도록 설계한 branching 규칙이 bound 중심 규칙보다 integer feasibility에 빨리 이른다는 것을 Pryor and Chinneck(2011)이 보였는데, BLS는 내내 그 영역에 있다. constraint programming에서는 value ordering을 해로부터(Chu and Stuckey, 2015) 또는 deep Q-learning으로(Cappart et al., 2021) 변수 선택과 함께 또는 별도로 학습했다.

### 2.3 Market split, lattice 방법, complete solver

Cornuéjols and Dawande(1999)의 market split 문제는 (1)과 같은 시스템에서 branch-and-bound의 약점을 드러내는 벤치마크다: n=10(m−1)개의 이진 변수 위에 등식 행 m개, 계수는 {0,⋯,99}에서 균등하게 뽑고 각 우변은 행 합의 절반으로 두므로, relaxation polytope가 넓고 vertex가 정수성에서 멀며 bounding은 거의 가지치지 못한다; branch-and-cut은 m≈7에서 멈춘다. Aardal et al.(2000)은 이 instance를 lattice 위에 재정식화해 풀었다: Aardal–Hurkens–Lenstra embedding은 해를 lattice의 가장 짧은 vector 중 하나로 만들고, LLL/BKZ reduction(Lenstra et al., 1982; Schnorr and Euchner, 1994)이 그것을 찾으며, 복원한 vector를 검증하므로 방법은 건전하다. lattice enumeration은 이 벤치마크에서 여전히 최고 수준이다: Wassermann(2025)은 QOBLIB market-split instance를 CPU 하나로 m=14까지 푼다. 우리 instance는 market split을 어렵게 만드는 재료 — 밀집한 행과 범위 한가운데의 우변 — 를 공유하고, 0/1 계수·심어진 해·최대 80개 변수의 탐색 영역이라는 점에서 다르다; 따라서 lattice reduction은 BLS solver의 자연스러운 전처리 단계다. clause learning을 갖춘 constraint solver(Marques-Silva and Sakallah, 1999; Ohrimenko et al., 2009)는 내려가면서 정보를 제조하며 우리가 비교하는 wall-clock 기준이다. 직접 satisfiability 예측은 기록이 약하고(Selsam et al., 2019; Li et al., 2023) message-passing network는 어떤 feasible/infeasible 쌍을 원리적으로 구분할 수 없다(Chen et al., 2023). 비싼 node별 계산을 대체하는 node별 추정치를 학습하는 것은 amortized inference(Amos, 2023; Gershman and Goodman, 2014)의 사례다.

### 2.4 이 접근들이 BLS에 남겨 둔 것

우리 설정에 가장 가까운 세 learned-branching 계열은 feasibility 시스템으로 옮겨오면 각각 본질적인 무언가를 잃는다. strong branching 모방(Khalil et al., 2016; Gasse et al., 2019)은 목적함수를 전제한다: expert는 두 자식의 bound 개선으로 후보를 점수 매기는데 BLS에는 개선할 bound가 없다. feasibility 시스템에서 strong branching이 보유하는 유일한 정보는 자식의 relaxation이 infeasible한지 여부이므로 expert는 LP probing — 반증할 수 있는 것을 고정하고 LP 값으로 분기하는 규칙 — 으로 퇴화하며 node마다 자유 변수당 relaxation 하나를 지불한다; 그것을 모방하면 node당 비용 없이 순위를 물려받겠지만 학습된 모방이 expert의 tree를 유지하는지는 열어 두는 질문이며(6절), 우리가 시험하는 것은 expert 그 자체다. message passing에 의한 근사 marginal(Li and Si, 2022; Montanari et al., 2007)은 BLS에서 잘 정의된다: 각 행은 cardinality factor이고 정확한 factor message를 쓰는 sum-product는 구현 가능하다. 두 가지가 잘못된다. factor graph가 밀집해 모든 행이 변수의 약 3분의 1에 닿으므로 loopy belief propagation은 강한 damping이 필요하고 수렴하는 곳에서도 수백 번 반복한다; 그리고 NSNet식 사용법은 root에서 한 번 marginal을 추정해 incomplete local search용 배정을 decode하는 반면 complete search는 모든 node에서 partial assignment에 **조건부**인 marginal이 필요하다. subtree 크기에 대한 from-scratch 강화학습(Etheve et al., 2020; Scavuzzo et al., 2022)은 depth-first search에 올바른 credit assignment를 제공한다. 제공하지 않는 것은 값 선택이다: 변수–값 쌍 위의 단일 policy는 어느 변수로 분기할지를 가르치는 같은 희박한 보상으로 어느 값을 먼저 시도할지도 배워야 하는데, BLS는 열거 가능한 instance에서 최적 값을 conditional marginal로 정확히 제공한다; 또한 학습된 순서가 어디서 출발해야 하는지, tree 크기의 이득을 어떻게 귀속시켜야 하는지도 말하지 않는다. 학습과 무관한 공백이 둘 더 있다. bounded enumeration을 갖춘 lattice reduction(Aardal et al., 2000; Wassermann, 2025)은 건전하며 우리가 다루는 모든 instance를 풀고, n=60에서는 1초 미만, n=80에서는 분 단위가 든다(5.5절); BLS solver의 자연스러운 전처리 단계이며 branching 규칙의 비교는 그런 단계를 끈 탐색만으로 이루어져야 한다. 그리고 network에 feasibility 판정을 맡길 수 없으므로(Selsam et al., 2019; Chen et al., 2023) 학습 규칙은 탐색의 순서만 정하고 탐색을 완전하게 남겨야 한다. 4절의 프레임워크는 이 두 제약 아래 세 학습 공백을 닫도록 만들어졌다: 문제가 제공하는 정확한 conditional marginal로 값을, 탐색이 산출하는 정확한 subtree 비용으로 변수를, 따로 그 순서로 학습하고, 결코 변수를 확정하거나 가지를 치지 않는다.

## 3. 문제 정의와 준비

### 3.1 Instance와 범위

우리는 (1)의 feasible instance를 다룬다: 모든 instance는 해를 심어 생성하므로 $S \neq \varnothing$이며 infeasibility 판별은 본 논문의 범위 밖이다. 구성은 2절의 market-split 방식에 0/1 계수를 적용한 것이다. 생성기는 각 원소가 확률 0.3으로 1인 무작위 A를 뽑고 정확히 n/2개가 1인 무작위 $\mathbf{x}^*$를 심어 $\mathbf{b}=A\mathbf{x}^*$로 두므로, 모든 우변은 행 범위의 한가운데, 즉 0/1 완성의 수가 가장 많고 propagation이 발동할 수 없는 곳에 놓인다. A마다 후보 planting 20개 중 relaxation polytope가 가장 넓은 것(무작위 선형 목적 4개가 돌려주는 vertex 간 평균 거리로 측정)을 남기므로 채택된 instance의 relaxation은 생성기가 만들 수 있는 만큼 무정보하다. 유일성은 강제하지 않는다; 열거가 완료되는 크기에서 해의 수 중앙값은 n=25, 50, 60에서 64, 112, 113이다.

## 3.2 Conditioning은 reduction이다

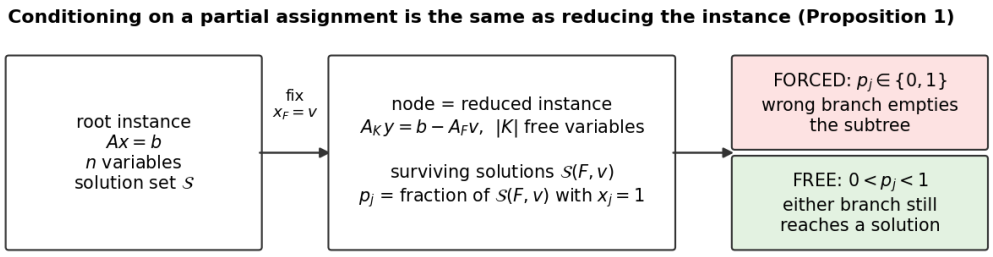



그림 2. 명제 1과 그것이 유도하는 분할. $\mathbf{x}_F=\mathbf{v}$인 node는 더 작은 BLS $A_K\mathbf{y}=\mathbf{b}-A_F\mathbf{v}$이고 그 conditional marginal은 그 instance의 통상적 marginal이다. 살아남은 모든 해가 일치하는 변수가 FORCED이며, 그 반대로 분기할 때만 backtrack 비용이 든다.

(1) 위의 depth-first search의 node는 고정 변수 집합 F 위의 partial assignment $\mathbf{x}_F=\mathbf{v}$이고 K는 자유 변수다. $S(F,\mathbf{v})=\{\mathbf{x}\in S: \mathbf{x}_F=\mathbf{v}\}$를 node와 일관된 해집합이라 하고

$$p_j = \Pr[x_j=1 \mid A, \mathbf{b}, \mathbf{x}_F=\mathbf{v}] = |\{\mathbf{x}\in S(F,\mathbf{v}): x_j=1\}| \,/\, |S(F,\mathbf{v})|, \; j\in K \quad (2)$$

를 conditional marginal, 즉 S 위의 균등 측도로 정의한 marginal이라 하자.

**명제 1.** $A_F$, $A_K$를 F, K로 인덱싱된 열 부분행렬이라 하면 $\mathbf{x}\to\mathbf{x}_K$는 $S(F,\mathbf{v})$에서 $\{\mathbf{y}\in\{0,1\}^{|K|}: A_K\mathbf{y}=\mathbf{b}-A_F\mathbf{v}\}$로의 전단사다. 따라서 원본 instance의 conditional marginal은 축소 instance의 통상적 marginal이다. 증명. $A\mathbf{x}=A_F\mathbf{x}_F+A_K\mathbf{x}_K$이고 $\mathbf{x}_F=\mathbf{v}$로 고정하면 $A\mathbf{x}=\mathbf{b} \Leftrightarrow A_K\mathbf{x}_K=\mathbf{b}-A_F\mathbf{v}$이며 $\mathbf{x}_F$가 결정되어 있으므로 전단사다. marginal 진술은 개수를 세면 따른다. □

두 귀결이 있다(그림 2). 첫째, 하나의 architecture·라벨 생성기·학습 루프가 모든 node를 담당한다: depth-d 학습 상태는 n−d개 변수의 instance로 저장되고, 작은 instance로 학습한 network는 더 큰 instance의 탐색이 만들어내는 축소 instance에서 평가된다. 둘째, $p_j\in\{0,1\}$은 살아남은 모든 해가 $x_j$에 일치한다는 뜻이고 반대로 분기하면 subtree가 빈다. 이런 변수를 FORCED, 나머지를 FREE라 부른다: FREE 변수는 어느 branch로도 해에 도달하므로 첫 값이 틀려도 해를 포함한 subtree를 탐색하는 것 이상의 비용이 없다. FORCED 오류만이 backtrack 비용을 낳는다.

## 3.3 Node의 모습과 정확도 상한

작은 instance에서는 $S(F,\mathbf{v})$를 열거할 수 있으므로 해의 어떤 predictor도 넘을 수 없는 Bayes 최적 변수별 정확도를 계산할 수 있다:

$$\mathrm{ceil}(F,\mathbf{v}) = (1/|K|)\,\Sigma_{j\in K}\,\max(p_j, 1-p_j) \quad (3)$$

이는 살아남은 해의 다수 값으로 답할 때 달성된다. 10×25 instance의 held-out in-tree 상태에서 살아남은 해집합은 root의 중앙값 74에서 depth 10이면 하나로 줄고, 자유 변수 중 FORCED 비중은 root의 3%에서 depth 5의 41%, depth 8의 73%, depth 12의 94%로 오른다. root에서는 거의 어떤 결정도 비용을 낳을 수 없고 상한 (3)은 FREE 변수에 지배된다; depth 6부터는 대부분의 결정이 비용을 낳을 수 있다. 그림 3은 그 상한에 대한 Stage 1 network를 depth별로 보여준다.

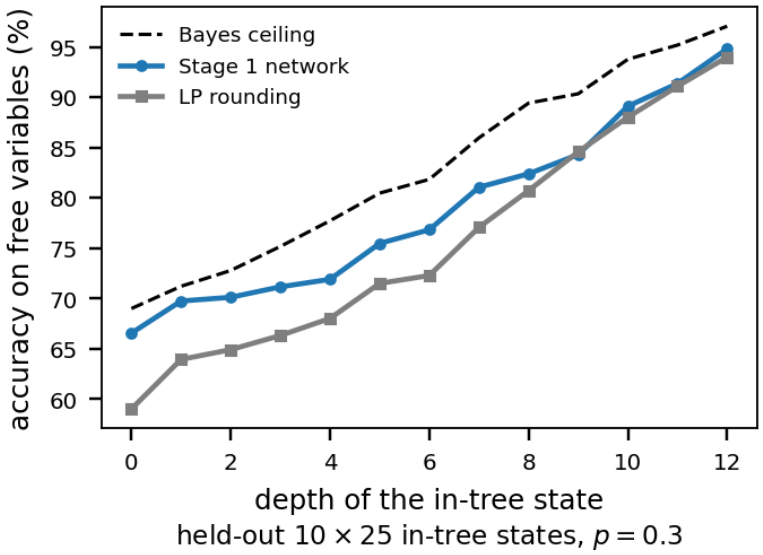



그림 3. 10×25 instance held-out in-tree 상태에서 정확한 marginal에 대한 예측, depth별: Stage 1 network와 relaxation rounding의 변수별 정확도를 Bayes 상한 (3)과 비교.

## 4. 제안 방법

### 4.1 개요

solver는 학습된 network가 branching 결정을 안내하는 depth-first search다(그림 1); 학습된 것은 결코 변수를 확정하거나 가지를 치지 않으므로 탐색은 완전하게 유지된다. network(그림 4)는 두 단계로 학습한다(그림 5): Stage 1은 열거한 instance의 정확한 conditional marginal에 marginal head를 맞추고, Stage 2는 동결된 trunk 위에 policy head를 더해 모든 branching 결정이 치르는 정확한 node 수로 fine-tuning한다. BKZ 뒤 bounded enumeration을 붙인 Aardal–Hurkens–Lenstra embedding의 lattice reduction(Aardal et al., 2000)은 우리가 다루는 모든 instance를 푸는 건전한 전처리 단계이며 작은 크기에서는 싸다(5.5절); 방법의 일부가 아니며 측정되는 차이가 branching 결정뿐이도록 모든 branching 규칙 비교에서 끈다.

### 4.2 Guided depth-first search

모든 instance는 알고리즘 1의 depth-first 절차로 처음부터 탐색한다. node는 partial assignment이며 명제 1에 따라 축소 instance $(A', \mathbf{b}') = (A_K, \mathbf{b} - A_F \mathbf{v})$로 다루고, $r_i$는 $A'$의 행 $i$에 남은 자유 변수 수다.

알고리즘 1. guided depth-first search의 node 하나. 1~4단계는 건전하며 변수를 확정하거나 가지를 치는 유일한 단계다; 학습 컴포넌트는 5단계에만 들어간다.

| 단계 | 알고리즘 1: guided depth-first search의 node 하나 |
|---|---|
| 1 | **Reduce.** $(F, \mathbf{v})$를 pop; $A' = A_K$, $\mathbf{b}' = \mathbf{b} - A_F \mathbf{v}$를 만든다. $r_i = 0$인 행은 버린다; 행이 남지 않으면 $A\mathbf{x} = \mathbf{b}$를 검증하고 $\mathbf{x}$를 반환한다. |
| 2 | **Bound check.** 어떤 행에서 $b'_i < 0$ 또는 $b'_i > r_i$이면 node를 버린다. |
| 3 | **Propagate.** $b'_i = 0$(그 행의 자유 변수를 0으로 고정) 또는 $b'_i = r_i$(1로 고정)인 행이 있는 동안 고정을 적용한다; 모순이면 node를 버리고, 변수가 고정됐으면 축소 node를 push하고 돌아간다. |
| 4 | **Relax.** $F$의 bound를 $\mathbf{v}$로 조인 단일 warm-start relaxation $\{A\mathbf{x} = \mathbf{b},\ 0 \le \mathbf{x} \le 1\}$을 재최적화한다(이전 basis에서 dual simplex, 0.08 ms). infeasible이면 node를 버리고, 아니면 vertex $\mathbf{x}^{LP}{}_K$를 읽는다. |
| 5 | **Predict.** $(A', \mathbf{b}', \mathbf{x}^{LP}{}_K)$의 bipartite graph 위에서 network를 한 번 forward하여 $j \in K$에 대한 $\hat{p}_j$와 $g_j$를 얻는다(4.5절의 dense 경로에서 0.3~0.5 ms). $j^* = \arg\max_j g_j$, $v = 1[\hat{p}_{j^*} \ge 1/2]$를 고른다. |
| 6 | **Branch.** 자식 $x_{j^*} = 1 - v$를 먼저, 자식 $x_{j^*} = v$를 나중에 push하여 $v$를 먼저, backtrack 시 $1 - v$를 탐색한다. |

세 가지 설계 선택이 이후에 중요하다. 0/1 행에서 propagation은 "이 행의 자유 변수 $r_i$개 중 정확히 $b'_i$개가 1"이라 읽히며 $b'_i \in \{0, r_i\}$일 때만 변수를 고정할 수 있으므로 우리 instance의 root 근처에서는 작동하지 않는다. relaxation은 node마다 다시 만들지 않는다: 원래 $n$개 변수 위의 linear program 하나를 탐색 내내 유지하고 node를

bound 변경 집합으로 표현하므로, 자식이든 backtrack 후의 형제든 연속한 node는 bound 몇 개만 다르고 dual simplex는 부모 basis에서 재개한다; 이는 재구성보다 50배 싸며 Python 루프가 초당 약 1,000 node를 처리하게 하는 요인이다. 그 infeasibility 판정은 행들의 선형결합을 보므로 propagation보다 엄격히 강하고 network 앞의 마지막 건전 단계다. 그것이 돌려주는 vertex $\mathbf{x}^{LP}$가 network의 주 입력 특징이다.

## 4.3 Stage 1: 정확한 conditional marginal을 target으로

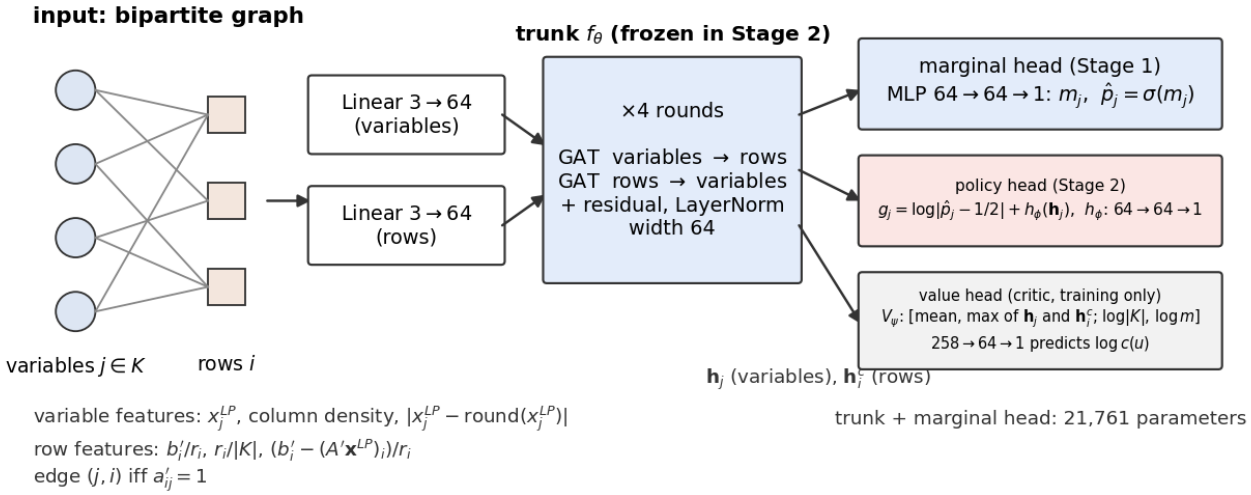



그림 4. network. 축소 instance가 변수당·행당 특징 3개를 지닌 bipartite graph로 들어가고, graph-attention 4 라운드의 trunk가 변수와 행마다 embedding을 만들며, head 셋이 그것을 읽는다. marginal head는 Stage 1에서, policy head는 동결된 trunk 위에서 Stage 2에서 학습하고, value head는 Stage 2의 critic으로만 쓰인다.

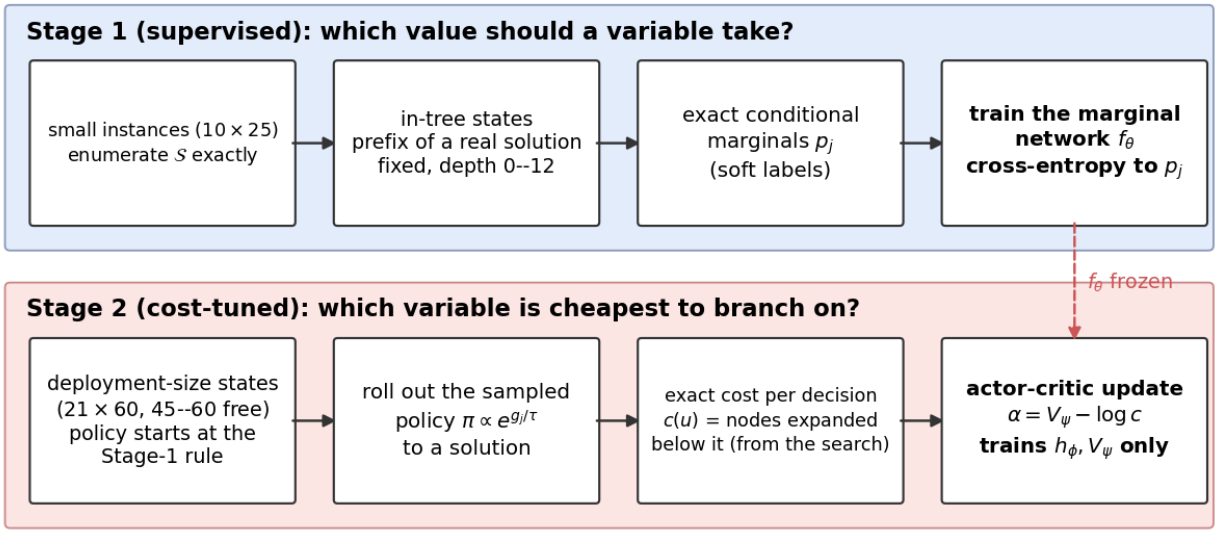



그림 5. 두 학습 단계. Stage 1은 열거 가능한 instance의 정확한 conditional marginal에서 **어느 값**을 배우고, Stage 2는 탐색이 산출하는 모든 결정의 정확한 node 수에서 **어느 변수**를 배운다. Stage 1 network는 동결된다.

### Network

network $f_\theta$(그림 4)는 graph attention(Velickovic et al., 2018)을 쓰는 bipartite 변수–제약 graph network로 Gasse et al.(2019)의 architecture 계열이다. 변수 node는 특징 3개(relaxation 값 $x^{LP}_j$, 열 밀도, fractionality $|x^{LP}_j - \text{round}(x^{LP}_j)|$), 제약 node는 3개(우변 $b'_i/r_i$, 밀도 $r_i/|K|$, relaxation 잔차 $(b'_i - (A'x^{LP})_i)/r_i$)를 지니고, $a'_{ij}=1$인 곳에 간선이 있다. linear layer가 둘을 폭 64로 올리고, attention 4 라운드가 변수→제약, 제약→변수 message를 번갈아 주고받으며(residual 연결, layer normalization), 2층 marginal head가 각 변수 embedding $\mathbf{h}_j$를 logit 하나로 사상한다: $\hat{p}_j = \sigma(m_j)$; 파라미터는 총 21,761개. Stage 2에서 더하는 head 둘은 4.4절에 있다. architecture는

표준이고 기여는 target이다.

### Target과 학습 상태

열거 가능한 instance(10×25)에서 S는 constraint solver의 all-solutions mode로 얻고 $p_j$는 개수로 계산한다. 소진까지 완료된 열거만 인정한다; 해를 이미 모은 채 시간제한에 걸린 열거는 잘린 S를 대입해 모든 라벨을 편향시킨다. 학습 상태는 도달 가능한 in-tree 상태다: depth $d \in \{0,\dots,12\}$를 균등하게 뽑고 LP 확신도 순서의 prefix를 무작위로 고른 실제 해의 값으로 고정한 뒤 축소 instance를 다시 열거해 자유 변수의 $p_j$를 얻는다. network는 soft target에 대한 cross-entropy

$$L_1 = -(1/|K|)\,\Sigma_{j\in K}\,[\,p_j \log \hat{p}_j + (1-p_j)\log(1-\hat{p}_j)\,] \quad (4)$$

로 학습하며 이는 $\hat{p}=p$에서 정확히 최소화된다; $p_j$ 자리에 심어진 해를 넣는 통상적 선택은 $|S|>1$이면 잡음 섞인 대리 target이다.

### Branching 규칙

node에서 network를 한 번 평가하고 Stage 1 규칙은 가장 확신하는 marginal의 다수 값으로 분기한다:

$$j^* = \operatorname{argmax}_{j\in K} |\hat{p}_j - 1/2|, \quad v = 1[\hat{p}_{j^*} \geq 1/2] \quad (5)$$

## 4.4 Stage 2: 정확한 subtree node 수를 목적함수로

규칙 (5)는 살아남은 해의 더 큰 몫이 취하는 값을 먼저 시도하여 첫 자식이 marginal이 약속하는 만큼의 해를 보존하게 한다. 첫 자식이 해를 포함하지 않을 때 탐색이 얼마를 지불하는지, 자식이 얼마나 작아지는지는 말하지 않는다; 둘 다 subtree의 성질이고 depth-first search는 그것을 정확히 산출한다. node u에 대해 c(u)를 u 아래에서 해를 찾거나 subtree를 소진할 때까지 전개한 node 수라 하자; 탐색은 u에서 backtrack해 나올 때 그것을 돌려준다. c(u)를 u에서 내린 branching 결정의 비용으로 쓴다; c(root)는 전체 탐색이 해에 도달하기까지 전개하는 node 수이며 실험 전체에서 보고하는 tree 크기 지표다. Stage 2는 $-\log c(u)$에 대해 branching 순서를 fine-tuning한다.

### 결정 과정

상태는 propagation 후의 축소 instance, 행동은 자유 변수 j, 첫 값은 동결된 Stage 1 network의 $1[\hat{p}_j \geq 1/2]$로 유지한다. 각 결정이 자기 subtree를 가지므로 credit은 episode가 아니라 결정 단위로 배정된다 — sparse한 해결/미해결 보상에는 없는 성질이며 문제를 well-posed하게 만드는 성질이다(Etheve et al., 2020).

### Policy와 value head

$f_\theta$의 동결된 trunk 위에 head 둘을 더한다(그림 4). policy logit은

$$g_j(s) = \log |\hat{p}_j - 1/2| + h_\phi(\mathbf{h}_j) \quad (6)$$

이며 $h_\phi$는 마지막 층을 0으로 초기화한 2층 network라서 $\pi(j|s) \propto \exp(g_j/\tau)$는 규칙 (5)를 부드럽게 한 것에서 출발하고 그 argmax는 정확히 그 규칙이다. value head $V_\psi(s)$는 mean·max pooling한 변수·제약 embedding과 $\log|K|$, $\log m$으로 $\log c$를 예측한다.

### 갱신

매 epoch 자유 변수 45~60개인 학습 상태 400개를 뽑아 표집 policy($\tau=1$)를 해까지 전개하고 모든 결정을 정확한 비용과 함께 기록한다(epoch당 약 21,000 결정). epoch 안에서 표준화한 advantage $\alpha(u)=V_\psi(s_u)-\log c(u)$로

$$L_2 = \Sigma_u\,[\,-\alpha(u)\log \pi(j_u|s_u) - \lambda\, H(\pi(\cdot|s_u)) + (V_\psi(s_u) - \log c(u))^2\,], \ \lambda=0.01 \quad (7)$$

이는 Monte-Carlo return을 쓰는 on-policy actor-critic 갱신이며(Williams, 1992) return이 정확하므로 bootstrapping이 필요 없다. $\phi$, $\psi$만 학습한다; $\theta$는 고정이므로 값을 고르고 FORCED/FREE 분할을 정의하는 marginal은 변하지 않고 어떤 개선도 branching 순서만으로 귀속된다. critic은 규칙 (5) 하에서 기록한 결정으로 사전학습하며 그 held-out 순위 상관이 actor 갱신의 관문이다 — 비용 순위를 못 매기는 baseline은 advantage를 잡음으로 만든다. epoch는 학습 풀의 held-out 상태 아래 총 node 수로 고르며 평가 instance는 결코 쓰지 않는다. 배포 시 policy는 greedy다: $j^*=\operatorname{argmax}_j g_j(s)$.

### 4.5 Node당 비용

자유 변수 |K|개인 node에서 guided search는 warm-start relaxation 하나와 forward pass 하나를 지불한다. forward pass는 dense adjacency tensor 위에서 TorchScript로 돈다(graph가 최대 변수 80개·행 28개라 sparse message passing은 얻는 것이 없다); 단일 thread로 자유 변수 수에 따라 0.3~0.5 ms이며 Stage 2의 head는 측정 가능한 비용을 더하지 않는다. 단일 relaxation 고전 규칙은 relaxation만 지불하므로 학습 규칙의 node는 1.3~1.5배 비싸며, 그 비율은 relaxation이 크기와 함께 커짐에 따라 줄어든다(5.2절, 표 3). 같은 FORCED 변수를 탐지하는 건전 절차인 LP probing($x_j$를 각 값으로 고정해 relaxation이 infeasible해지는 쪽을 가지치기)은 node마다 relaxation |K|개를 지불한다.

## 5. 실험

실험은 1절의 네 질문에 설정 다음에 차례로 답한다(5.2~5.5절).

### 5.1 설정

#### Instance

instance는 3.1절대로 생성한다. 크기는 10×25, 18×50, 21×60, 24×70, 28×80이고 제약 밀도 $m/n \approx 0.35$를 유지한다; 평가 세트는 작은 두 크기에서 30개, 큰 세 크기에서 각 100개이며 생성기 seed로 학습 풀과 분리된다. Stage 1은 별도의 10×25 instance 1,600개의 in-tree 상태로, Stage 2는 별도의 21×60 instance 500개의 상태로 학습한다. n=100에서는 CP-SAT로 20 s 안에 어떤 방법도 instance를 풀지 못하므로 n=80이 마지막 크기다.

#### 학습

Stage 1: 소진까지 열거한 10×25 instance 1,600개, instance당 depth 0~12의 in-tree 상태 8개, 학습 상태 10,000개·테스트 상태 2,000개; Adam, 학습률 $10^{-3}$, 20 epoch, batch 1, gradient clipping 1.0, CPU 코어 하나로 약 20분, 튜닝한 hyperparameter 없음, validation split 없음. 테스트 상태의 FORCED 변수 정확도는 79.1%로 Bayes 상한 83.3%, relaxation rounding 76.0~77.5%와 비교된다. Stage 2: 자유 변수 45~60개인 21×60 상태에서 Stage 1 규칙으로 기록한 결정으로 critic을 사전학습하고(log c와의 held-out Spearman 상관 0.81, 상태 크기만 아는 baseline 0.75), 갱신 (7)을 6 epoch; epoch는 held-out 학습 상태로 고른다(epoch 6, 그 상태 아래 node 수가 Stage 1 규칙의 0.36).

표 1. 비교하는 branching 규칙. 모두 같은 depth-first 루프(알고리즘 1: node당 warm-start relaxation 하나, 같은 propagation, 같은 예산)에서 돌고 5단계만 다르다. 단일 relaxation 고전 규칙은 node당 relaxation 하나, LP probing은 자유 변수당 하나를 지불한다; 참조 신호는 배포 불가능하며 값 신호가 할 수 있는 것의 상한이다.

| 그룹 | 규칙 | 분기 변수 | 첫 값 |
|---|---|---|---|
| 고전 | LP rule (most-integral) | $\operatorname{argmax}_j \lvert x^{LP}_j - 1/2 \rvert$ | $\operatorname{round}(x^{LP}_j)$ |
| | most-fractional | $\operatorname{argmin}_j \lvert x^{LP}_j - 1/2 \rvert$ | $\operatorname{round}(x^{LP}_j)$ |
| | maxSD (Pesant et al., 2012) | 행 안 solution density 최대 | 밀한 값 |
| | dom/wdeg (Boussemart et al., 2004) | 가중 실패 차수 최대 | $\operatorname{round}(x^{LP}_j)$ |
| | impact (Refalo, 2004) | 평균 탐색공간 축소 최대 | impact가 작은 값 |
| | LP probing (= BLS의 strong branching) | 반증 가능한 값을 고정한 뒤 LP rule | $\operatorname{round}(x^{LP}_j)$ |
| | uniform | 균등 무작위 자유 변수 | 균등 |
| 참조 | oracle marginals | $\operatorname{argmax}_j \lvert p_j - 1/2 \rvert$, $p_j$는 열거로 정확 | $1[p_j \geq 1/2]$ |
| | BP marginals (NSNet-based) | 위와 같되 damped sum-product의 $\tilde{p}_j$ | $1[\tilde{p}_j \geq 1/2]$ |
| 학습 | Stage 1 | 가장 확신하는 marginal, $\operatorname{argmax}_j \lvert \hat{p}_j - 1/2 \rvert$ | $1[\hat{p}_j \geq 1/2]$ |
| | Stage 1+2 | 비용 조정 점수, $\operatorname{argmax}_j g_j$ | $1[\hat{p}_j \geq 1/2]$ |

#### 규칙

표 1이 규칙을 나열한다. LP 규칙 둘은 most-integral과 most-fractional vertex 규칙이다; maxSD는 counting-based search로 0/1 행의 solution density가 정확히 $b'_i/r_i$라 계산 비용이 없다; dom/wdeg와 impact는

constraint programming의 표준 적응형 규칙 둘로 우리 루프의 실패·축소 통계로 구현했다; LP probing은 목적함수 없는 strong branching이 환원되는 것이다; uniform 규칙은 instance가 어렵다는 점검이다. oracle은 모든 node에서 $S(F,v)$를 열거하며 감당 가능한 곳에서만 돌린다; BP 규칙은 정확한 cardinality-factor message, damping 0.9, 최대 300회 반복의 sum-product를 돌린다. 2절의 학습 baseline은 다음과 같이 나타난다: LP probing은 strong branching 모방을(expert 그 자체다), BP는 message-passing marginal을, 5.4절에서 무작위 head와 LP 규칙에서 출발한 비용 조정 policy는 from scratch와 고전 규칙에서 출발하는 강화학습을 대표한다.

**프로토콜과 통계**

instance마다 두 지표를 기록한다: 해에 도달하기까지 전개한 node 수(4.4절의 $c$(root); branching 품질을 재며 탐색이 relaxation vertex가 주어지면 결정적이므로 seed 불변)와 단일 thread wall-clock 시간(node당 비용도 반영). 예산은 다섯 크기에 대해 instance당 60, 300, 600, 1,200, 1,200 s이며 프로세스 종료로 강제한다; 20코어 Intel Core Ultra 7 265KF 머신(메모리 32 GB)에서 instance 6개를 동시에 돌리고 GPU는 쓰지 않으며 학습 arm은 4.5절의 dense TorchScript 경로를 쓴다. 루프는 Python 3.10과 NumPy 2.2, relaxation은 LP 엔진으로 쓴 Gurobi 13.0, network는 PyTorch 2.10이고, solver 비교는 PySCIPOpt 6.1을 통한 SCIP 10.0, OR-Tools CP-SAT 9.15, Gurobi 13.0, fpylll 0.6을 쓴다. 비교는 instance별로 짝짓는다: 두 규칙이 모두 푼 instance에서 instance별 비율의 중앙값, 그 중앙값의 95% bootstrap 구간(instance 1,000회 재표집), 각 규칙이 이긴 instance 수, 동률을 제외한 양측 exact sign test를 보고하며, PAR-2(미해결 instance를 예산의 두 배로 센 평균 시간)가 해결률과 시간을 함께 요약한다. node·시간 원값의 중앙값은 방향을 보여주기 위한 것이고 검정은 그 위에서 하지 않는다. 다중 비교 보정은 하지 않는다; 논문 전체에 짝 검정이 약 60개 있으므로 0.05 근처의 개별 p값은 시사적인 것으로 읽어야 하고 결론은 $p<10^{-3}$인 비교에 기댄다. 학습 arm의 타이밍은 부록 D에서 평가 seed 두 개로 더 반복한다.

## 5.2 RQ1: 고전 branching과 전이된 learned branching 대비

표 2. 크기당 100개 instance에서 같은 탐색 루프의 branching 규칙. 중앙값은 해결 instance 기준; 마지막 두 열은 공통 해결 instance에서 규칙 대 Stage 1+2의 instance별 비율 중앙값, 95% bootstrap 구간, 양측 sign test p값(동률 제외). 비율이 1보다 크면 규칙이 학습 규칙보다 node를 더 쓰거나 더 오래 걸린다. PAR-2는 미해결 instance를 예산의 두 배로 센다.

| 규칙 | 해결 | node 중앙값 | 초 중앙값 | PAR-2 s | node, 규칙 / Stage 1+2 | 시간, 규칙 / Stage 1+2 |
|---|---|---|---|---|---|---|
| 21×60 (100), budget 600 s | | | | | | |
| LP rule (most-integral) | 100/100 | 7,762 | 4.58 | 8.0 | 23.90× [15.11, 37.29] $10^{-25}$ | 10.55× [7.68, 18.48] $10^{-22}$ |
| most-fractional | 100/100 | 662 | 0.41 | 0.7 | 2.27× [1.29, 4.39] $10^{-4}$ | 1.13× [0.79, 1.57] 0.48 |
| maxSD | 100/100 | 494 | 0.30 | 0.7 | 1.52× [1.13, 2.45] $10^{-3}$ | 0.77× [0.61, 1.32] 0.48 |
| dom/wdeg | 100/100 | 436 | 0.25 | 0.5 | 1.52× [1.08, 1.95] 0.04 | 0.83× [0.61, 0.96] 0.02 |
| impact | 100/100 | 1,464 | 2.68 | 5.8 | 5.76× [2.69, 7.72] $4 \cdot 10^{-9}$ | 7.06× [3.98, 12.84] $10^{-15}$ |
| LP probing | 100/100 | 130 | 1.99 | 2.9 | 0.37× [0.28, 0.44] $10^{-6}$ | 5.36× [4.47, 6.32] $10^{-22}$ |
| uniform | 100/100 | 2,112 | 1.21 | 2.0 | 5.12× [3.66, 9.31] $10^{-14}$ | 2.81× [1.98, 4.26] $10^{-10}$ |
| Stage 1 | 100/100 | 628 | 0.65 | 1.2 | 2.32× [1.08, 3.25] 0.01 | 2.02× [1.19, 2.83] $10^{-3}$ |
| **Stage 1+2** | 100/100 | **325** | **0.36** | 0.6 | — | — |
| 24×70 (100), budget 1200 s | | | | | | |
| LP rule (most-integral) | 100/100 | 95,440 | 51.95 | 81.5 | 37.74× [27.17, 66.01] $8 \cdot 10^{-27}$ | 21.88× [16.38, 40.52] $8 \cdot 10^{-27}$ |
| most-fractional | 100/100 | 6,504 | 3.97 | 6.8 | 3.67× [2.13, 4.69] $6 \cdot 10^{-11}$ | 2.43× [1.42, 2.94] $4 \cdot 10^{-4}$ |
| maxSD | 100/100 | 5,637 | 3.23 | 5.4 | 2.46× [1.71, 3.10] $10^{-7}$ | 1.46× [1.10, 1.87] $4 \cdot 10^{-3}$ |
| dom/wdeg | 100/100 | 5,263 | 2.73 | 3.9 | 1.79× [1.15, 2.42] 0.04 | 1.09× [0.68, 1.37] 0.48 |
| impact | 100/100 | 16,256 | 33.07 | 64.7 | 8.92× [5.57, 10.95] $8 \cdot 10^{-14}$ | 19.01× [12.89, 25.36] $6 \cdot 10^{-24}$ |
| LP probing | 100/100 | 1,088 | 14.27 | 23.4 | 0.36× [0.24, 0.57] $10^{-3}$ | 5.17× [3.40, 7.94] $6 \cdot 10^{-11}$ |
| uniform | 100/100 | 16,028 | 9.70 | 18.0 | 6.76× [4.55, 9.10] $10^{-14}$ | 4.15× [3.02, 5.50] $6 \cdot 10^{-11}$ |
| Stage 1 | 100/100 | 8,174 | 6.60 | 9.1 | 3.17× [2.32, 4.40] $10^{-6}$ | 2.63× [2.10, 3.85] $5 \cdot 10^{-6}$ |
| **Stage 1+2** | 100/100 | **2,580** | **2.39** | 3.6 | — | — |
| 28×80 (100), budget 1200 s | | | | | | |
| LP rule (most-integral) | 40/100 | 1,029,039 | 622.27 | 1688.2 | 19.25× [13.83, 27.87] $10^{-9}$ | 14.60× [10.28, 22.08] $10^{-9}$ |
| most-fractional | 99/100 | 187,381 | 114.49 | 221.3 | 3.98× [2.41, 5.74] $7 \cdot 10^{-9}$ | 3.07× [1.96, 4.63] $9 \cdot 10^{-7}$ |
| maxSD | 100/100 | 141,178 | 88.12 | 135.8 | 3.20× [2.29, 4.79] $10^{-9}$ | 2.39× [1.70, 3.51] $5 \cdot 10^{-6}$ |
| dom/wdeg | 100/100 | 152,598 | 81.46 | 119.3 | 3.15× [2.47, 4.17] $10^{-7}$ | 1.96× [1.57, 2.65] $10^{-3}$ |
| impact | 49/100 | 149,624 | 357.32 | 1428.4 | 2.50× [1.69, 3.50] $10^{-3}$ | 7.21× [4.97, 10.09] $6 \cdot 10^{-8}$ |
| LP probing | 78/100 | 17,148 | 309.62 | 819.8 | 0.35× [0.21, 0.47] $6 \cdot 10^{-7}$ | 7.71× [4.65, 10.17] $10^{-15}$ |
| uniform | 91/100 | 520,855 | 289.99 | 537.4 | 7.28× [5.93, 9.77] $7 \cdot 10^{-18}$ | 5.02× [4.08, 6.86] $10^{-13}$ |
| Stage 1 | 90/100 | 212,271 | 182.65 | 485.0 | 5.01× [3.01, 6.62] $9 \cdot 10^{-11}$ | 5.30× [3.22, 6.89] $10^{-11}$ |
| **Stage 1+2** | 100/100 | **58,026** | **46.61** | 74.5 | — | — |

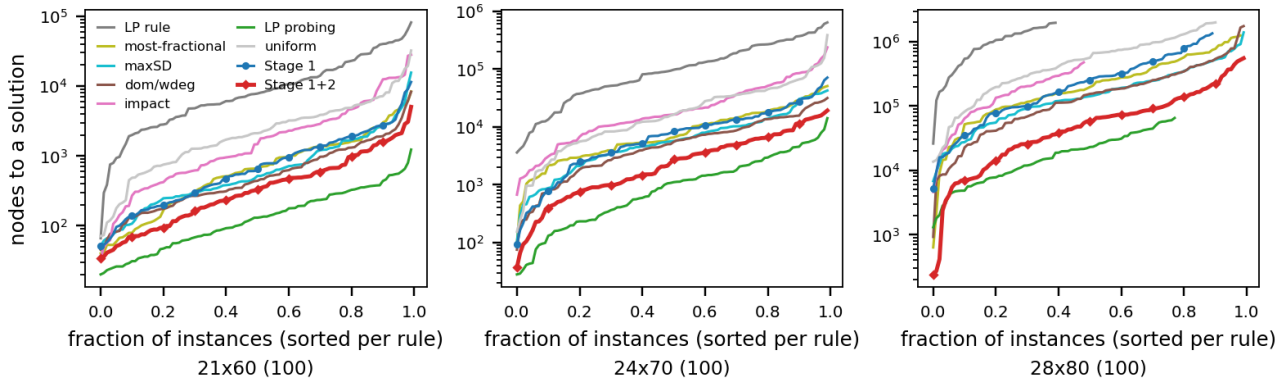



그림 6. 세 100개 세트에서 해에 도달하기까지의 node 수, 규칙마다 instance를 따로 정렬; 다른 곡선 아래의 곡선은 모든 분위에서 더 작은 tree다. node마다 자유 변수당 relaxation 하나를 쓰는 LP probing만이 학습 규칙 아래에 있다.

### Tree 크기

표 2와 그림 6이 주 결과다. 세 크기 각각에서 학습 규칙은 단일 relaxation 고전 규칙 전부보다 node를 적게 쓰고 짝 검정은 그 전부에 대해 유의하다: 크기별 최강의 싼 규칙(21×60과 24×70에서 dom/wdeg, 28×80에서 maxSD) 대비 instance별 비율 중앙값은 1.52배(p=0.035), 1.79배(p=0.035), 3.20배($p=10^{-9}$); pseudocost가 없을 때 MIP solver가 쓰는 most-fractional branching 대비 2.27배, 3.67배, 3.98배($p \le 2\times10^{-4}$); most-integral LP rule 대비 작은 두 크기에서 24배와 38배이고, 28×80에서 그 규칙은 100개 중 40개만 예산 안에 풀며 그 위에서 19배 많은 node를 쓴다(PAR-2 1,688 대 75 s). 이득은 분포적이지 보편적이지 않다: 학습 규칙은 싼 규칙들에 대해 100개 중 61~80개에서 이기고 나머지에서 진다. 더 작은 tree를 만드는 유일한 규칙은 LP probing으로, 푸는 instance에서 2.7~2.9배이며 strong branching의 전이형이다: 모든 node에서 relaxation이 반증할 수 있는 모든 변수를 고정하므로 어떤 점수보다 node당 정보가 많고, node당 relaxation |K|개를 지불한다. Stage 1 단독(비용 조정 순서 없는 marginal 규칙)은 세 크기에서 전체 규칙보다 node를 2.3배, 3.2배, 5.0배 더 쓰고(짝 중앙값), 그 자체로는 21×60과 24×70 모두에서 LP rule보다 약 12배 아래이며, 21×60에서 싼 고전 규칙과 동급이고 24×70부터는 그 뒤이며 28×80 instance 10개를 풀지 못한다: marginal만으로는 경쟁력 있는 값 신호이자 평범한 변수 점수이고, 규칙을 앞서게 하는 것은 비용 조정 순서다.

표 3. dense 추론 경로의 node당 비용(단일 relaxation 행은 LP rule, 즉 most-integral branching이며 표 1의 모든 단일 relaxation 고전 규칙이 같은 node당 비용을 가진다): 크기당 instance 6개, 각 규칙의 전체 tree를 계측(node당 ms와 warm-start relaxation·network forward pass가 차지하는 node 시간 비중의 instance 간 중앙값; 나머지는 축소·propagation·장부 처리). forward pass 한 번은 0.33~0.53 ms이며 자유 변수 수에 따라 커진다.

| 크기 | 규칙 | ms / node | LP 비중 | forward 비중 |
|---|---|---|---|---|
| 21×60 | single-relaxation rule (LP rule) | 0.47 | 90% | 0% |
| 21×60 | Stage 1 | 0.67 | 64% | 27% |
| 21×60 | hlStage 1+2 | hl0.70 | 57% | 32% |
| 24×70 | single-relaxation rule (LP rule) | 0.55 | 91% | 0% |
| 24×70 | Stage 1 | 0.75 | 69% | 23% |
| 24×70 | hlStage 1+2 | hl0.73 | 62% | 28% |
| 28×80 | single-relaxation rule (LP rule) | 0.61 | 92% | 0% |
| 28×80 | Stage 1 | 0.86 | 69% | 23% |
| 28×80 | hlStage 1+2 | hl0.82 | 62% | 29% |

### Wall-clock

작은 tree는 node당 비용이 허락할 때만 시간을 사며, 표 3이 전체 tree를 계측해 그 비용을 잰다. 단일 relaxation 규칙의 node는 세 크기에서 0.47, 0.55, 0.61 ms이고 그 90%가 warm-start relaxation이다; 학습 규칙의 node는 0.71, 0.73, 0.82 ms이며 forward pass가 28~32%, relaxation이 57~62%다. 따라서 overhead 비율은 n=60에서 1.5배, n=70, 80에서 1.3배인데 forward pass가 relaxation보다 크기에 따라 느리게 커지기 때문이고, 그동안 node 우위는 1.5배에서 3.2배로 오르며 wall-clock이 그 뒤를 따른다(표 2). 21×60에서 학습 규칙은 most-fractional·maxSD branching과 같은 속도이고 dom/wdeg보다 느리다(0.83배, p=0.021); 24×70에서는 most-fractional(2.4배, $p=4\times10^{-4}$)과 maxSD(1.5배, p=0.004)보다 빠르고 dom/wdeg와 동률이다; 28×80에서는 전부보다 3.1배, 2.4배, 2.0배 빠르고($p \le 0.002$), most-fractional 99개·uniform 91개 대비 100/100을 풀며 PAR-2가 가장 낮다. LP probing은 node당 relaxation을 자유 변수당 하나씩 13~18 ms 지불하여 더 작은 tree에도 세 크기에서 학습 규칙보다 5~8배 느리고, 28×80에서 instance 22개를 풀지 못한다(PAR-2 820 대 75 s). 역시 probing하는 impact-based search는 21×60에서 7배, 24×70에서 19배 느리면서 node를 6~9배 쓴다(짝 중앙값). 작은 두 크기(10×25, 18×50; 30개, tree 14~120 node)에서는 모든 규칙이 수백분의 1초 안에 있고 학습 규칙은 10×25에서 가장 느린데 14-node tree는 forward pass 14번을 상각할 수 없기 때문이다; 이 크기들은 부록 A에 보고하며 질문이 결정되는 곳이 아니다.

**참조 신호**

표 1의 두 규칙은 배포 불가능하며 값 신호가 할 수 있는 것의 상한이다. 모든 node에서 S(F,**v**)를 열거하고 가장 확신하는 정확한 marginal로 분기하는 oracle은 10×25에서 node 22개를 쓰며 Stage 1 network 21개, 비용 조정 규칙 14개, dom/wdeg 15.5개와 비교된다: 확신도 규칙 아래의 정확한 값 신호는 학습된 근사보다 나을 것이 없고 고전적 변수 점수보다 나쁘므로, 남은 여지는 Stage 2가 다루는 변수 선택에 있다. NSNet 전이형인 belief-propagation marginal은 18×50에서 Stage 1과 같은 크기의 tree(118 대 116 node)를 추정 한 번에 35 ms, dense forward pass 0.3~0.5 ms의 비용으로 만들어 학습 규칙 wall-clock의 50배다; marginal network는 이 신호를 훨씬 싼 값에, 그리고 이 비용의 message passing으로는 불가능한 node당 한 번씩 돌리는 것이다. NSNet 자체의 marginal 사용법, 즉 root 추정 하나를 incomplete local search의 초기 배정으로 decode하는 방식도 돌렸다: 잔차 $||A\mathbf{x}-\mathbf{b}||_1$에 대한 WalkSAT식 탐색은 21×60의 100개를 전부 1.2~1.5 s에 풀어 학습 탐색의 3~4배 시간이 들며, 초기 배정이 marginal network·LP 반올림·균등 추출 어느 것에서 오든 같은 시간이 걸리므로 이 크기에서 marginal은 local search에 아무것도 사 주지 않고 비교는 신호의 두 사용법이 아니라 두 탐색 사이의 것이다.

## 5.3 RQ2: 더 큰 instance로의 일반화

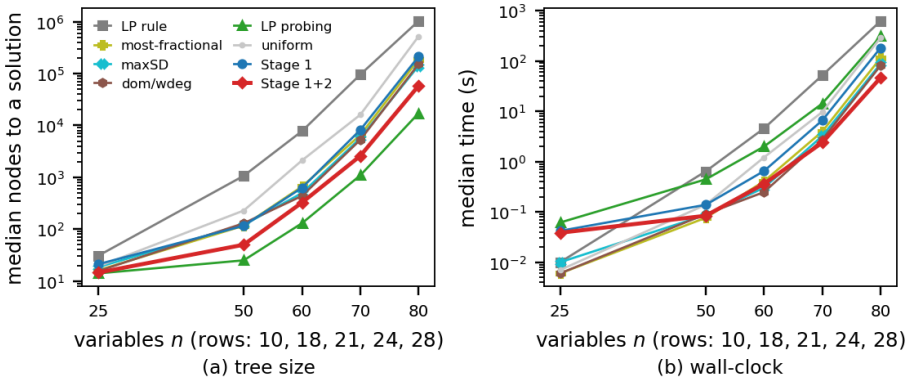



그림 7. 크기에 따른 스케일링. (a) 규칙별 node 중앙값과 (b) wall-clock 중앙값(n=25, 50은 30개; n=60, 70, 80은 100개). Stage 1은 n=25, Stage 2는 n=60에서 학습했으므로 n=70, 80은 외삽이다.

두 단계 모두 중요한 크기 아래에서 학습했다: marginal network는 10×25 상태, branching 순서는 21×60 상태이며, architecture는 크기와 무관하고(축소 instance 위의 graph network) 학습 신호는 어느 크기에서도 정의된다. 작은 instance에서 탐색 heuristic을 학습해 더 큰 instance에 배포하는 것은 operations research의 학습 탐색에서 통상적인 체제이며(Hottung et al., 2020; Yan et al., 2025), 질문은 그 우위가 외삽을 견디는가이다. 그림 7과 표 2가 두 부분으로 답한다. 첫째, 규칙은 전이된다: 어느 단계도 본 적 없는 24×70과 28×80에서 모든 instance를 예산 안에 풀고 단일 relaxation 규칙 전부에 대해 유의한 node 우위를 유지한다. 둘째, 우위는 줄지 않고 크기와 함께 커진다: 최강의 싼 규칙 대비 node 비율은 n=60의 1.5배에서 70의 1.8배, 80의 3.2배로, most-fractional branching 대비 2.3배에서 3.7배, 4.0배로 오르고, dom/wdeg 대비 wall-clock 비율은 n=60(0.83배)과 70(1.09배) 사이에서 1을 넘어 80에서 2.0배에 이른다. 기제는 표 3의 것이다: forward pass의 node당 premium은 고정인데 relaxation과 tree는 커지므로 상수 배의 node 우위는 tree가 충분히 크면 시간 우위가 되고, node 우위 자체도 탐색이 긴 곳에서 더 크며 5.4절은 이를 비용 조정 순서가 틀린 값을 반증하는 방식으로 추적한다. 전이는 단계 사이에도 고르지 않다: Stage 1 단독은 모든 크기에서 LP rule 대비 node 우위를 유지하지만 24×70부터 시간에서, 28×80에서 해결률에서(90/100) 싼 고전 규칙에 뒤지므로 유용하게 전이되는 것은 조합이며 Stage 2의 Stage 1 대비 이득은 n=60의 2.3배에서 n=80의 5.0배로 커진다. 전이가 하지 못하는 것은 유용한 창을 넓히는 일이다: n=100에서는 어떤 규칙도 예산 안에 instance를 풀지 못하며 다음 크기에는 더 싼 node나 clause learning이 필요하다.

## 5.4 RQ3: 두 신호는 대체 가능한가, 각 단계는 필요한가?

표 4. 값×변수 교차 ablation: 각 arm은 변수 선택 점수 하나와 첫 값 규칙 하나를 결합하고 나머지는 동일하다. 크기별 열: node 중앙값과 Stage 1+2 대비 instance별 node 비율 중앙값(bootstrap 구간, sign test p값).

| 변수, 값 | node (21×60) | 비율 | node (24×70) | 비율 | node (28×80) | 비율 |
|---|---|---|---|---|---|---|
| LP var., LP value (LP rule) | 7,762 | 23.90× [15.11, 37.29] $10^{-25}$ | 95,440 | 37.74× [27.17, 66.01] $8\cdot10^{-27}$ | 1,029,039 | 19.25× [13.83, 27.87] $10^{-9}$ |
| LP var., S1 value | 9,810 | 27.87× [19.53, 36.44] $10^{-21}$ | 69,872 | 39.45× [20.99, 58.92] $10^{-21}$ | 647,145 | 16.32× [6.86, 23.92] $9\cdot10^{-5}$ |
| S1 var., S1 value (Stage 1) | 628 | 2.32× [1.08, 3.25] 0.01 | 8,174 | 3.17× [2.32, 4.40] $10^{-6}$ | 212,271 | 5.01× [3.01, 6.62] $9\cdot10^{-11}$ |
| S1 var., LP value | 956 | 2.84× [1.77, 3.75] $5\cdot10^{-6}$ | 6,998 | 3.28× [2.56, 4.22] $6\cdot10^{-7}$ | 278,175 | 5.85× [4.04, 8.77] $5\cdot10^{-12}$ |
| S2 var., S1 value (Stage 1+2) | 325 | — | 2,580 | — | 58,026 | — |
| S2 var., LP value | 292 | 0.79× [0.60, 1.04] 0.09 | 2,348 | 1.19× [0.91, 1.66] 0.27 | 69,665 | 1.41× [1.03, 1.86] 0.09 |

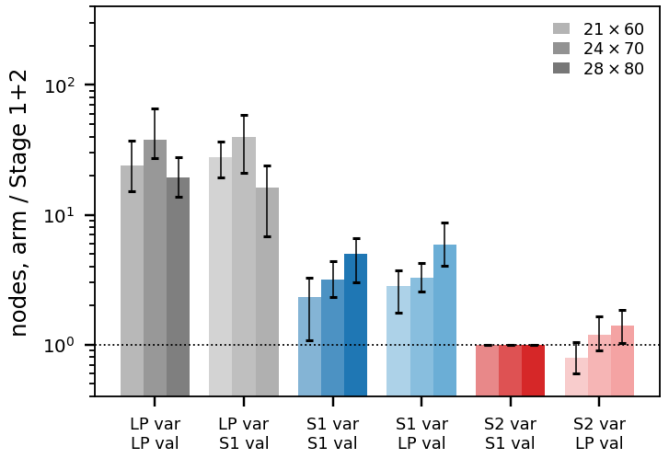



그림 8. 표 4의 교차 ablation을 Stage 1+2 대비 instance별 node 비율 중앙값으로(log 눈금, 95% bootstrap 구간), 세 크기.

### 값과 변수는 분리 가능하며 이득은 변수가 지닌다

표 4와 그림 8은 변수 선택 점수 셋(LP most-integral, Stage 1 확신도, Stage 2 비용 조정)과 첫 값 규칙 둘(LP 반올림, Stage 1 marginal)을 각각 결합한다. 표를 열로 읽으면 변수 점수가 결정적이다: Stage 1 값을 고정한 채 변수 점수를 LP에서 Stage 1, Stage 2로 옮기면 21×60에서 tree가 12배, 다시 2.3배 줄고, 정확히 학습한 marginal 값을 붙인 LP 변수는 전체 규칙보다 28배, 24×70에서 37배 나쁘며 전체 규칙이 전부 푸는 28×80에서 100개 중 26개를 푼다. 행으로 읽으면 값 규칙은 그렇지 않다: Stage 2 변수를 고정한 채 marginal 값을 LP 반올림으로 바꾸면 tree는 21×60에서 0.79배(p=0.09), 24×70에서 1.19배(p=0.27), 28×80에서 1.41배(p=0.09)로 어느 것도 5% 수준에서 유의하지 않되 tree가 커질수록 방향은 marginal 값 쪽으로 기울며, Stage 1 변수를 고정하면 LP 값이 작은 두 크기에서 1.5배와 1.3배 더 많은 node를 들이고 그 arm은 크기와 함께 전체 규칙에서 더 멀어지는데(2.8배, 3.3배, 5.9배) 이것이 tree와 함께 커지는 Stage 2의 이득이다. 두 결론이 따르며 하나는 4절의 설계에 반한다. 신호는 중요한 의미에서 대체 불가능하다: marginal로 학습한 network는 trunk와 Stage 2가 출발하는 확신도 점수를 통해 좋은 변수 점수를 가능하게 하는 것이고, 어떤 값 정확도도 LP 변수 선택을 구제하지 못한다. 그러나 정확한 marginal 값은 대체 가능하다: 비용 조정 점수로 변수를 고르고 나면 relaxation을 반올림해도 어느 크기에서도 유의하게 다르지 않은, 28×80에서 기껏 1.4배 큰 tree가 나오며, 이는 확신도 규칙 아래의 정확한 marginal조차 고전적 변수 점수를 이기지 못한다는 5.2절의 oracle 결과와 일치한다. 이 방법에서 값 신호의 역할은 값을 고르는 것이 아니라 표현을 학습시키는 것이다.

### Stage 2가 배운 것

fine-tuning은 instance별 기록이 드러내는 방식으로 탐색을 바꾼다(부록 B의 표 A2). 모든 규칙에 대해 해 경로에서 첫 번째로 시도한 값이 반증된(그 아래 subtree가 소진된) 빈도와 그 반증 한 번의 비용을 기록한다. Stage 1 확신도 규칙은

가장 드물게 틀리고(21×60에서 결정의 6%) 실수당 가장 많이 지불한다(246 node); 비용 조정 규칙은 세 배 자주 틀리고(19%) 실수당 4분의 1을 지불하며(65 node) 고르는 변수가 더 많은 propagation과 relaxation 고정을 촉발하므로 해 경로가 더 짧다(결정 46개 대 28개). 24×70의 고전 규칙에 대해서도 같다: dom/wdeg, maxSD, most-fractional branching은 결정의 14~16%에서 각 835~911 node로 반증되고 학습 규칙은 21%에서 387 node다. 이는 설계가 아니라 비용 신호에서 학습된 fail-first(Haralick and Elliott, 1980)다: policy는 가장 확신하는 변수에서 벗어나 틀린 값이 싸게 소진되는 변수로 옮겨 가며, Stage 2 변수에 LP 값을 붙인 arm도 같은 양상(16%, 63 node)이므로 구조는 변수 선택에서 온다.

표 5. 학습 변형: 기준 Stage 1+2 규칙 대비 node 중앙값과 instance별 node 비율 중앙값, 크기별. Stage 1 변형은 확신도 규칙으로 돌린다(Stage 2 없음); Stage 2 변형은 따로 명시하지 않는 한 동결된 기준 trunk를 공유한다.

| 변형 | node (21×60) | 비율 | node (24×70) | 비율 | node (28×80) | 비율 |
|---|---|---|---|---|---|---|
| Stage 1 (reference) | 628 | 2.32× [1.08, 3.25] 0.01 | 8,174 | 3.17× [2.32, 4.40] $10^{-6}$ | 212,271 (90/100) | 5.01× [3.01, 6.62] $9\cdot10^{-11}$ |
| Stage 1, hard 0/1 labels | 848 | 2.66× [1.93, 4.05] $3\cdot10^{-5}$ | 7,644 | 2.73× [2.03, 3.98] $4\cdot10^{-9}$ | 259,605 (95/100) | 4.18× [3.26, 6.63] $10^{-12}$ |
| Stage 1, training seed 1 | 562 | 1.72× [1.17, 2.49] $4\cdot10^{-3}$ | 7,318 | 2.72× [1.60, 4.60] $4\cdot10^{-4}$ | 207,117 (91/100) | 4.03× [2.63, 5.13] $10^{-10}$ |
| Stage 1, training seed 2 | 553 | 1.65× [1.10, 2.14] 0.04 | 6,199 | 2.06× [1.01, 2.76] 0.09 | 213,055 (95/100) | 4.37× [3.09, 6.74] $3\cdot10^{-11}$ |
| Stage 1, 2,771 states (size ctrl.) | 662 | 2.22× [1.47, 2.61] $8\cdot10^{-5}$ | 6,897 | 3.89× [1.96, 6.15] $3\cdot10^{-5}$ | 168,442 (91/100) | 4.12× [3.08, 6.44] $10^{-9}$ |
| Stage 1 on 21×60 states (M1) | 912 | 2.44× [1.64, 4.10] $10^{-5}$ | 10,754 | 3.94× [2.51, 7.91] $10^{-7}$ | 225,182 (85/100) | 5.73× [2.99, 7.19] $8\cdot10^{-8}$ |
| Stage 1, NSNet architecture | 892 | 2.75× [2.02, 4.03] $10^{-7}$ | 12,500 | 5.02× [3.46, 7.46] $5\cdot10^{-13}$ | 248,556 (70/100) | 4.75× [3.45, 7.73] $10^{-8}$ |
| Stage 1+2 (reference) | 325 | — | 2,580 | — | 58,026 | — |
| Stage 2 from a random head | 333 | 1.04× [0.74, 1.28] 0.92 | 1,878 | 0.86× [0.65, 1.10] 0.27 | 59,122 (99/100) | 1.03× [0.94, 1.20] 0.84 |
| Stage 2 from the LP rule | 376 | 1.06× [0.73, 1.31] 0.76 | 3,024 | 0.86× [0.62, 1.39] 0.48 | 72,539 | 1.12× [0.87, 1.48] 0.37 |
| Stage 2, trunk unfrozen | 280 | 0.76× [0.56, 1.20] 0.37 | 2,427 | 0.83× [0.69, 1.12] 0.37 | 59,692 | 1.06× [0.70, 1.65] 0.76 |

### 각 단계는 필요한가, 레시피의 어느 부분이 중요한가?

표 5는 나머지를 고정한 채 각 단계의 학습을 바꾼다. Stage 1에서는 레시피의 어느 것도 값 정확도를 움직이지 않는다: 라벨 형태(심어진 0/1 해 대 soft marginal), 학습 seed, 학습 상태 수(10,000 대 2,771)는 FORCED 변수의 테스트 정확도를 기껏 0.2 point 바꾸고(상한 83.3% 대비 79.0~79.2%), 같은 상태로 학습한 NSNet architecture (Li and Si, 2022)의 network는 80.4%에 이르며, 10×25 대신 21×60 상태로 학습하면(M1) 21×60 상태의 정확도가 2.4 point 오른다. tree는 다른 이야기를 한다. 기준 레시피의 학습 seed 둘은 21×60에서 기준의 628 대비 553과 562 node를 쓰므로 Stage 1 규칙에는 약 ±10%의 seed 분산이 있고 어떤 라벨·architecture 선택도 올바른 방향으로 그것을 넘지 못한다; 값 정확도가 **더 높은** 두 network인 M1과 NSNet architecture가 **가장 큰** tree를 쓰며(21×60에서 912와 892; 24×70에서 기준 레시피의 6,199~8,175 대비 10,754와 12,500) 28×80에서는 예산 안에 가장 적게 푼다(100개 중 85와 70; 다른 Stage 1 변형 91~95, 전체 규칙 100); 모든 Stage 1 변형이 거기서 전체 규칙의 4.0~5.7배 node를 쓴다. 따라서 값 정확도와 tree 크기는 같은 양이 아니며 이는 표 4를 반대편에서 본 요점이고, Stage 1 레시피가 필요한 것은 이 network들 중 어느 것이든 Stage 2가 2~3배 개선하는 trunk와 확신도 점수를 공급한다는 의미에서만이다. Stage 2에서는 레시피가 우리가 설계한 것보다 덜 중요하다. policy head를 무작위로 시작하면 — 이 trunk로 from-scratch 강화학습이 할 일 — 초기 policy는 marginal 규칙의 7배 root 비용에서 출발하지만 6 epoch 뒤에는 기준과 동급이고(세 크기에서 1.04배, 0.86배, 1.03배, 모두 비유의), LP 규칙에서 시작해도 같은 곳에 이른다(1.06배, 0.86배, 1.12배). trunk를 풀면 held-out node 수가 셋째 epoch부터 발산하지만(Stage 1 규칙의 2.7~5.4배) epoch 선택이 남기는 첫 epoch의 checkpoint는 동결된 것만큼 좋다(0.76배, 0.83배, 1.06배, 비유의). 따라서 동결된 trunk는 fine-tuning을 계속할 수 있을 만큼 안정하게 유지하고 이득을 순서에 귀속시키는 것이지 tree 크기의 원천이 아니며, 우리는 그렇게 주장하지 않는다.

## 5.5 RQ4: Complete solver와의 결합

표 6. SCIP 안(presolve·cut·primal heuristic 끔, depth-first node 선택, 첫 해에서 정지): SCIP 자체 규칙 대 branching plug-in으로 꽂은 학습 규칙, conflict analysis off/on. 비율은 표 2와 같이 규칙 / plug-in.

| conflict | 규칙 | 해결 | node 중앙값 | 초 중앙값 | node, 규칙 / plug-in | 시간, 규칙 / plug-in |
|---|---|---|---|---|---|---|
| 21×60 (100) | | | | | | |
| off | RPB (SCIP default) | 100/100 | 828 | 0.06 | 2.14× [1.57, 3.42] $5\cdot10^{-6}$ | 0.34× [0.24, 0.51] $6\cdot10^{-7}$ |
| off | most-fractional (SCIP) | 100/100 | 558 | 0.05 | 1.83× [1.37, 2.39] $10^{-4}$ | 0.26× [0.20, 0.39] $10^{-15}$ |
| off | inference (SCIP) | 100/100 | 767 | 0.06 | 2.41× [1.77, 3.80] $8\cdot10^{-5}$ | 0.32× [0.23, 0.45] $6\cdot10^{-11}$ |
| off | **Stage 1+2 plug-in** | 100/100 | **274** | **0.14** | — | — |
| 21×60 (100) | | | | | | |
| on | RPB (SCIP default) | 100/100 | 705 | 0.06 | 1.86× [1.46, 2.81] $3\cdot10^{-5}$ | 0.32× [0.25, 0.46] $10^{-9}$ |
| on | most-fractional (SCIP) | 100/100 | 528 | 0.04 | 1.69× [1.33, 2.27] $10^{-4}$ | 0.28× [0.23, 0.39] $10^{-11}$ |
| on | inference (SCIP) | 100/100 | 795 | 0.07 | 2.09× [1.62, 2.84] $10^{-5}$ | 0.33× [0.24, 0.45] $10^{-12}$ |
| on | **Stage 1+2 plug-in** | 100/100 | **256** | **0.14** | — | — |
| 24×70 (100) | | | | | | |
| off | RPB (SCIP default) | 100/100 | 7,010 | 0.56 | 3.47× [2.42, 6.00] $6\cdot10^{-11}$ | 0.83× [0.58, 1.37] 0.37 |
| off | most-fractional (SCIP) | 100/100 | 5,026 | 0.40 | 3.47× [1.59, 4.23] $3\cdot10^{-5}$ | 0.86× [0.41, 0.98] 0.04 |
| off | inference (SCIP) | 100/100 | 6,694 | 0.50 | 4.00× [2.48, 6.36] $10^{-9}$ | 0.95× [0.60, 1.37] 0.48 |
| off | **Stage 1+2 plug-in** | 100/100 | **1,915** | **0.61** | — | — |
| 24×70 (100) | | | | | | |
| on | RPB (SCIP default) | 100/100 | 5,496 | 0.49 | 3.06× [2.57, 4.53] $6\cdot10^{-7}$ | 0.80× [0.64, 1.05] 0.13 |
| on | most-fractional (SCIP) | 100/100 | 5,340 | 0.50 | 2.98× [1.67, 3.87] $10^{-5}$ | 0.74× [0.48, 1.03] 0.13 |
| on | inference (SCIP) | 100/100 | 5,578 | 0.48 | 3.12× [2.26, 4.34] $10^{-9}$ | 0.78× [0.58, 0.97] 0.02 |
| on | **Stage 1+2 plug-in** | 100/100 | **1,806** | **0.60** | — | — |
| 28×80 (100) | | | | | | |
| off | RPB (SCIP default) | 100/100 | 224,088 | 25.05 | 3.83× [2.36, 5.96] $10^{-10}$ | 1.11× [0.71, 1.73] 0.48 |
| off | most-fractional (SCIP) | 100/100 | 136,840 | 15.84 | 2.11× [1.63, 3.01] $7\cdot10^{-3}$ | 0.66× [0.49, 0.93] 0.02 |
| off | inference (SCIP) | 100/100 | 232,221 | 25.19 | 3.90× [3.00, 5.70] $10^{-10}$ | 1.14× [0.84, 1.59] 0.62 |
| off | **Stage 1+2 plug-in** | 100/100 | **60,174** | **22.42** | — | — |
| 28×80 (100) | | | | | | |
| on | RPB (SCIP default) | 100/100 | 148,872 | 19.75 | 2.43× [1.80, 4.76] $3\cdot10^{-5}$ | 0.77× [0.57, 1.43] 0.37 |
| on | most-fractional (SCIP) | 100/100 | 128,310 | 17.10 | 2.08× [1.30, 2.76] $10^{-3}$ | 0.68× [0.41, 0.96] 0.06 |
| on | inference (SCIP) | 100/100 | 122,144 | 14.62 | 2.36× [1.52, 3.00] $10^{-4}$ | 0.71× [0.46, 0.90] 0.01 |
| on | **Stage 1+2 plug-in** | 100/100 | **58,564** | **24.00** | — | — |

### SCIP 안의 branching 규칙으로

학습 규칙은 축소 instance와 그 relaxation vertex의 함수이므로 MIP solver의 branching 규칙으로 설치할 수 있다. 우리는 PySCIPOpt branching plug-in으로 구현하고 SCIP(Bestuzheva et al., 2021)를 같은 instance에서 presolve·cutting plane·primal heuristic을 끄고 depth-first node 선택과 첫 해에서의 정지로 돌려, SCIP의 탐색이 우리 루프가 하는 것과 같되 propagation과 LP만 SCIP의 것이 되게 했다; 우리 루프에 없는 conflict analysis는 끄고 켜서 모두 돌린다. 비교 대상은 reliability pseudocost branching(RPB; SCIP 기본값이자 learning-to-branch 문헌이 모방하는 규칙, Achterberg et al., 2005), SCIP의 most-fractional 규칙, inference 규칙이다. conflict analysis는 SCIP의 clause learning이다: node가 relaxation이나 propagation으로 infeasible로 판정되면 solver는 dual ray나 propagation 이유에서 함께 성립할 수 없는 bound 변경의 최소 집합을

추출해 전역 conflict constraint로 더하고, 이후 같은 조합에 이르는 모든 subtree는 탐색 없이 잘린다. 우리 루프에는 그런 장치가 없으므로 끄고 켜서 모두 돌린다. 표 6은 node 우위가 엔진 교체와 clause learning 추가를 견딤을 보인다: 100개 세트에서 plug-in은 conflict analysis off/on 모두 모든 크기에서 RPB보다 node를 1.9~3.8배 적게 쓰고(sign test 전부 $p \le 3\times10^{-5}$) SCIP의 most-fractional·inference 규칙보다 1.7~4.0배 적게 쓴다. clause learning은 두 규칙을 불균등하게 돕는다: 켜면 RPB는 24×70에서 22%, 28×80에서 34% 적은 node를 쓰고 plug-in은 6%와 3%만 줄어, RPB 대비 node 비율은 3.5~3.8배에서 1.9~3.1배로 줄되 유의하게 남는다. 틀린 값을 이미 얕고 싸게 반증하는 규칙은 다른 곳에서 재사용할 긴 conflict를 적게 만들고, bound 중심 규칙은 conflict constraint가 잘라낼 것을 더 남긴다는 읽기다. 시간은 RPB와는 동률, 가장 싼 규칙에는 패배다: forward pass가 branching callback의 60~80%라 tree가 수백 node인 21×60에서 plug-in은 모든 SCIP 규칙보다 3배 느린 반면, 24×70과 28×80에서는 conflict analysis off/on 모두 중앙 시간이 RPB의 0.8~1.1배 안에 있고(짝 차이 어느 것도 비유의) SCIP most-fractional 규칙에는 0.66~0.86배로 뒤진다(p=0.02~0.06). 따라서 solver 안에서 RQ1의 답은 갈린다: 학습 규칙은 모든 크기에서 tree 크기로는 더 나은 branching 규칙이고, tree가 forward pass를 상각할 만큼 크면 solver 기본 규칙과 시간이 같으며, 가장 싼 규칙을 시간에서 이기지는 못한다.

표 7. end-to-end, 같은 instance·예산, 단일 thread: 기본 설정의 complete solver, bounded enumeration을 갖춘 lattice reduction, 학습 탐색. 해결 instance의 시간 중앙값과 PAR-2; node 수는 엔진 간 비교 불가라 생략한다.

| 방법 | 해결 | 초 중앙값 | PAR-2 s |
|---|---|---|---|
| 21×60 (100) | | | |
| SCIP (all defaults) | 100/100 | 0.35 | 0.5 |
| CP-SAT (1 worker) | 100/100 | 0.04 | 0.1 |
| Gurobi (1 thread) | 100/100 | 0.04 | 0.1 |
| BKZ-40 × 30 + enumeration | 100/100 | 0.20 | 0.4 |
| **Stage 1+2 (our loop)** | 100/100 | **0.36** | 0.6 |
| 24×70 (100) | | | |
| SCIP (all defaults) | 100/100 | 2.33 | 2.5 |
| CP-SAT (1 worker) | 100/100 | 0.36 | 0.6 |
| Gurobi (1 thread) | 100/100 | 0.31 | 0.4 |
| BKZ-40 × 30 + enumeration | 100/100 | 13.20 | 11.5 |
| **Stage 1+2 (our loop)** | 100/100 | **2.39** | 3.6 |
| 28×80 (100) | | | |
| SCIP (all defaults) | 100/100 | 16.47 | 21.1 |
| CP-SAT (1 worker) | 100/100 | 12.24 | 39.8 |
| Gurobi (1 thread) | 100/100 | 6.61 | 9.8 |
| BKZ-40 × 30 + enumeration | 100/100 | 124.90 | 117.7 |
| **Stage 1+2 (our loop)** | 100/100 | **46.61** | 74.5 |

### End-to-end

표 7은 학습 탐색을 기본 설정의 complete solver(thread 하나의 Gurobi, Gurobi Optimization, 2024; worker 하나의 CP-SAT, Perron and Didier, 2023; presolve·cut·heuristic·conflict analysis·RPB를 갖춘 SCIP)와 bounded enumeration을 갖춘 lattice reduction(BKZ block 40, 열 순열 30개, 300 s 캡의 fpylll enumeration, FPLLL team, 2023) 옆에 같은 instance와 예산으로 놓는다. 모든 방법이 모든 instance를 푼다. Gurobi가 모든 크기에서 가장 빠르고(n=60, 70, 80에서 중앙 0.05, 0.31, 6.6 s) CP-SAT가 그 다음이며(0.04, 0.36, 12.2 s; 28×80에서 꼬리가 무겁다), 기본 설정 SCIP는 24×70에서 학습 탐색과 동률(2.33 대 2.39 s)이고 28×80에서 2.8배 빠르다(16.5 대 46.6 s). 두 MIP solver 사이의 2.5~8배 격차는 우리 설정 탓이 아니다: 둘 다 thread 하나에서 기본 설정으로 돌며, 차이는 탐색이 빨리 닫히는 instance에서 효과를 보는 LP 엔진과 탐색 구현에서 온다(SCIP 자체 통계에서 presolve와 primal heuristic은 이 instance에서 아무것도 찾지 못한다); 표 6의 plug-in 비교는 presolve와 cut을 끈 SCIP라 다른 측정이다. lattice enumeration은 21×60 세트를 instance당 0.2 s에 풀지만 비용이 탐색보다 빠르게 커진다: 24×70에서 13.2 s, 28×80에서 125 s로 학습 탐색보다 5.5배와 2.7배 느리다. 두

가지가 동시에 참이다. 학습 탐색은 이 instance를 푸는 가장 빠른 방법이 아니다: 상용 MIP solver와 presolve·clause learning을 갖춘 CP-SAT solver는 n≤70에서 한두 자릿수, n=80에서 4~7배 앞서고 SCIP는 가장 큰 크기에서 앞선다. 그리고 그것은 branching 질문과 무관하다: branching 결정을 분리하는 비교는 표 2가며, SCIP 안에서 결정 품질은 그대로 옮겨지고 node당 비용은 그렇지 않다. 정직한 요약은 본 논문이 자기 루프와 SCIP 안에서 branching 규칙을 개선하지만 그 자체로 더 빠른 solver를 만들지는 않는다는 것이다; solver와의 격차는 clause learning과 compiled node이고 학습 규칙은 그 어느 것도 다루지 않는다.

## 6. 결론

branching 결정은 두 질문을 던지며 본 논문은 각각의 정확한 답으로 학습했다: 해를 열거할 수 있는 곳에서 계산 가능하고 reduction 항등식으로 모든 node에 옮겨지는 conditional marginal은 어느 값을 시도할지에, 탐색이 backtrack하면서 산출하는 결정 아래 node 수는 어느 변수로 분기할지에 답한다. 고전 규칙 7종과 learned-branching 세 계열의 전이형에 대해, 한 탐색 루프에서, 100개 세트로, 학습 규칙은 모든 크기에서 단일 relaxation 규칙 중 node를 가장 적게 쓰고 28×80에서는 가장 빠르기도 하다; n≤60에서 학습한 그 우위는 줄지 않고 크기와 함께 커진다; 그리고 node 우위를 SCIP 안으로 가져가 conflict analysis on/off의 reliability branching에 맞선다. ablation은 레시피가 무엇인지 말한다: 변수 선택이 이득을 지니고, 비용 조정 순서는 설계가 아니라 학습된 fail-first이며, 동결된 marginal trunk는 fine-tuning을 안정하게 유지하는 것이고, 정확한 marginal 값은 변수를 잘 고르고 나면 LP 반올림으로 대체 가능하며 가장 큰 크기에서 기껏 1.4배 차이다.

세 한계가 결과를 제한한다. 주장은 하나의 instance 계열, 즉 3.1절의 심어진 0/1 market-split 구성에 대한 것이며, 다른 제약 계열이나 실제 스캐너 구조의 투영 행렬에서도 성립하는지는 검증하지 않았다. 학습 탐색은 end-to-end로 경쟁력이 없다: 기본 설정의 Gurobi·CP-SAT·SCIP가 n≤70에서 자릿수 차이로 더 빠르므로 기여는 solver가 아니라 branching 규칙에 있고, lattice enumeration도 우리가 다루는 모든 instance를 풀되 큰 두 크기에서는 학습 탐색보다 느리다. 그리고 정확한 지도는 Stage 1을 열거되는 크기로 묶으므로 값 신호는 배포되는 곳에서 학습될 수 없다; LP 값이 그 역할을 하므로 그럴 필요가 없다는 것이 이 한계를 덜어 주는 한 결과다.

다음 단계는 부정적 결과에서 따른다. 값 신호의 역할이 표현을 학습시키는 것이라면 정확한 marginal보다 싼 target으로도 될 수 있고, Stage 2의 비용 신호는 열거가 전혀 필요 없으므로 열거가 실패하는 크기에서, 그리고 결정 아래 node 수는 subtree에 해가 있든 없든 정의되므로 본 논문이 제외한 infeasible instance에서 순서를 fine-tuning할 수 있다. solver 안의 시간 격차는 forward pass 예산이다: plug-in은 network가 callback의 60~80%인 채 시간에서 지므로, 더 작은 network나 node의 부분집합에서만 평가하는 학습 규칙이 두 지표 모두에서 이기는 규칙으로 가는 길이다.

## 부록 A. 작은 두 크기

표 A1. 작은 두 크기(각 30개): 표 2와 같은 열. tree는 수십 node이며 어떤 규칙도 점수의 비용 외에는 시간에서 갈라지지 않는다.

| 규칙 | 해결 | node 중앙값 | 초 중앙값 | PAR-2 s | node, 규칙 / Stage 1+2 | 시간, 규칙 / Stage 1+2 |
|---|---|---|---|---|---|---|
| 10×25 (30), budget 60 s | | | | | | |
| LP rule (most-integral) | 30/30 | 30 | 0.01 | 0.0 | 1.89× [1.53, 2.23] $7\cdot10^{-9}$ | 0.23× [0.20, 0.28] $10^{-9}$ |
| most-fractional | 30/30 | 16 | 0.01 | 0.0 | 1.03× [0.83, 1.18] 0.70 | 0.16× [0.13, 0.17] $10^{-9}$ |
| maxSD | 30/30 | 18 | 0.01 | 0.0 | 1.24× [1.18, 1.46] $10^{-5}$ | 0.27× [0.20, 0.30] $10^{-9}$ |
| dom/wdeg | 30/30 | 16 | 0.01 | 0.0 | 1.03× [0.93, 1.19] 0.85 | 0.16× [0.12, 0.18] $10^{-9}$ |
| impact | 30/30 | 17 | 0.03 | 0.0 | 1.14× [1.07, 1.28] 0.02 | 0.75× [0.67, 0.89] $6\cdot10^{-5}$ |
| LP probing | 30/30 | 14 | 0.06 | 0.1 | 0.94× [0.81, 1.07] 0.34 | 1.43× [1.28, 1.63] $10^{-4}$ |
| uniform | 30/30 | 19 | 0.01 | 0.0 | 1.23× [1.10, 1.41] $10^{-4}$ | 0.19× [0.15, 0.21] $10^{-9}$ |
| oracle marginals | 30/30 | 22 | 0.19 | 0.2 | 1.47× [1.39, 1.63] $10^{-9}$ | 4.52× [4.14, 5.41] $10^{-9}$ |
| BP marginals (NSNet-based) | 30/30 | 21 | 0.41 | 0.4 | 1.47× [1.32, 1.63] $4\cdot10^{-9}$ | 10.06× [9.01, 11.44] $10^{-9}$ |
| Stage 1 | 30/30 | 21 | 0.04 | 0.0 | 1.45× [1.30, 1.55] $4\cdot10^{-9}$ | 1.10× [1.05, 1.14] 0.02 |
| **Stage 1+2** | 30/30 | **14** | **0.04** | 0.0 | — | — |
| 18×50 (30), budget 300 s | | | | | | |
| LP rule (most-integral) | 30/30 | 1,066 | 0.64 | 0.9 | 13.89× [7.69, 42.45] $10^{-9}$ | 4.71× [3.68, 12.60] $9\cdot10^{-7}$ |
| most-fractional | 30/30 | 116 | 0.08 | 0.1 | 2.23× [1.04, 3.83] 0.10 | 0.82× [0.62, 1.12] 0.36 |
| maxSD | 30/30 | 120 | 0.09 | 0.1 | 2.18× [1.07, 3.26] 0.04 | 0.93× [0.74, 1.33] 0.58 |
| dom/wdeg | 30/30 | 130 | 0.09 | 0.1 | 2.39× [1.27, 4.62] 0.02 | 0.90× [0.52, 1.89] 1.00 |
| impact | 30/30 | 315 | 0.49 | 0.7 | 4.93× [3.26, 8.41] $9\cdot10^{-7}$ | 5.36× [3.57, 8.12] $6\cdot10^{-8}$ |
| LP probing | 30/30 | 25 | 0.44 | 0.5 | 0.55× [0.44, 0.70] $6\cdot10^{-5}$ | 5.36× [4.21, 6.34] $10^{-9}$ |
| uniform | 30/30 | 228 | 0.14 | 0.3 | 3.86× [2.12, 8.05] $3\cdot10^{-4}$ | 1.55× [0.94, 2.72] 0.10 |
| BP marginals (NSNet-based) | 30/30 | 118 | 4.12 | 6.1 | 1.85× [1.12, 3.22] 0.04 | 50.13× [34.89, 64.65] $10^{-9}$ |
| Stage 1 | 30/30 | 116 | 0.14 | 0.2 | 1.88× [1.01, 2.93] 0.06 | 1.40× [1.24, 2.08] $5\cdot10^{-3}$ |
| **Stage 1+2** | 30/30 | **50** | **0.08** | 0.1 | — | — |

표 A1은 10×25와 18×50을 보고한다. 참조 신호가 여기에 나타나는 것은 oracle이 모든 node에서 S(F,v)를 열거해 10×25에서만 감당 가능하고 belief propagation이 18×50에서 예산 안에 끝나기 때문이다.

## 부록 B. 모든 규칙의 반증 구조

표 A2는 모든 규칙과 모든 해결 instance에 대해 탐색이 node를 어떻게 썼는지 기록한다. 경로는 해에 이르는 branching 결정 수, 반증 비중은 해 없이 소진된 subtree에 속한 node의 비율, 첫 자식 실패율은 첫 값이 반증된 경로 결정의 비율(값 선택이 틀린 빈도), 반증 비용은 그런 틀린 첫 자식이 소진되기까지 든 평균 node 수, 깊이 비율은 그 틀린 결정들의 깊이 중앙값을 경로 길이로 나눈 것이다. 세 가지 읽기가 5.4절의 논증을 떠받친다. 첫째, 모든 규칙이 21×60에서 node의 90% 이상, 28×80에서 99% 이상을 반증된 subtree에 쓰므로 규칙은 얼마나 드물게 틀리느냐가 아니라 얼마나 싸게 반증하느냐로 평가된다. 둘째, Stage 1 규칙은 가장 드물게 틀리고 실수당 가장 많이 지불하는 규칙이며, 비용 조정 규칙은 2~3배 자주 틀리고 3~5배 적게 지불한다; Stage 2 변수에 LP 값을 붙인 arm도 같은 양상이므로 변화는 변수 선택에서 온다. 셋째, 비용 조정 규칙의 해 경로는 모든 크기에서 단일 relaxation 규칙 중 가장 짧은데 고르는 변수가 더 많은 propagation과 relaxation 고정을 촉발하기 때문이고, LP probing은 변수를 직접 고정하므로 더 짧다.

표 A2. 규칙별 반증 구조(해결 instance; instance 간 중앙값): node, 해 경로의 결정 수, 반증된 subtree의 node 비중, 첫 값이 반증된 경로 결정의 비율, 그런 반증 한 번의 평균 비용, 반증된 결정의 깊이 중앙값(경로 길이 대비).

| 규칙 | n | node | 경로 | 반증 비중 | 첫 자식 실패율 | 반증 비용 | 깊이 비율 |
|---|---|---|---|---|---|---|---|
| 21×60 (100) | | | | | | | |
| LP rule | 100 | 7,762 | 41 | 0.995 | 0.111 | 1,677 | 0.42 |
| most-fractional | 100 | 662 | 40 | 0.940 | 0.135 | 121 | 0.27 |
| maxSD | 100 | 494 | 36 | 0.917 | 0.156 | 89 | 0.59 |
| dom/wdeg | 100 | 436 | 34 | 0.920 | 0.113 | 115 | 0.42 |
| impact | 100 | 1,464 | 31 | 0.973 | 0.412 | 98 | 0.54 |
| LP-probing | 100 | 130 | 17 | 0.809 | 0.125 | 44 | 0.80 |
| uniform | 100 | 2,112 | 40 | 0.981 | 0.432 | 129 | 0.56 |
| Stage 1 | 100 | 628 | 46 | 0.923 | 0.064 | 246 | 0.52 |
| S1 var / LP val | 100 | 956 | 46 | 0.947 | 0.088 | 239 | 0.45 |
| S2 var / LP val | 100 | 292 | 25 | 0.909 | 0.160 | 63 | 0.37 |
| LP var / S1 val | 100 | 9,810 | 42 | 0.996 | 0.125 | 1,998 | 0.48 |
| 24×70 (100) | | | | | | | |
| LP rule | 100 | 95,440 | 51 | 0.999 | 0.119 | 15,884 | 0.34 |
| most-fractional | 100 | 6,504 | 49 | 0.992 | 0.151 | 911 | 0.26 |
| maxSD | 100 | 5,637 | 44 | 0.991 | 0.163 | 835 | 0.53 |
| dom/wdeg | 100 | 5,263 | 42 | 0.992 | 0.141 | 876 | 0.38 |
| impact | 100 | 16,256 | 40 | 0.997 | 0.421 | 1,078 | 0.52 |
| LP-probing | 100 | 1,088 | 20 | 0.974 | 0.163 | 296 | 0.73 |
| uniform | 100 | 16,028 | 48 | 0.997 | 0.437 | 846 | 0.56 |
| Stage 1 | 100 | 8,174 | 56 | 0.993 | 0.070 | 2,136 | 0.46 |
| S1 var / LP val | 100 | 6,998 | 55 | 0.992 | 0.103 | 1,319 | 0.45 |
| S2 var / LP val | 100 | 2,348 | 30 | 0.986 | 0.200 | 398 | 0.36 |
| LP var / S1 val | 100 | 69,872 | 50 | 0.999 | 0.134 | 9,734 | 0.45 |
| 28×80 (100) | | | | | | | |
| LP rule | 40 | 1,029,039 | 59 | 1.000 | 0.126 | 137,471 | 0.28 |
| most-fractional | 99 | 187,381 | 58 | 1.000 | 0.173 | 21,053 | 0.21 |
| maxSD | 100 | 141,178 | 52 | 1.000 | 0.206 | 13,784 | 0.47 |
| dom/wdeg | 100 | 152,598 | 50 | 1.000 | 0.179 | 17,945 | 0.27 |
| impact | 49 | 149,624 | 49 | 1.000 | 0.432 | 7,945 | 0.49 |
| LP-probing | 78 | 17,148 | 22 | 0.998 | 0.222 | 3,594 | 0.60 |
| uniform | 91 | 520,855 | 57 | 1.000 | 0.477 | 20,198 | 0.51 |
| Stage 1 | 90 | 212,271 | 65 | 1.000 | 0.079 | 41,686 | 0.41 |
| S1 var / LP val | 89 | 278,175 | 65 | 1.000 | 0.121 | 38,914 | 0.33 |
| S2 var / LP val | 100 | 69,665 | 37 | 0.999 | 0.269 | 7,026 | 0.33 |
| LP var / S1 val | 26 | 647,145 | 60 | 1.000 | 0.150 | 75,366 | 0.41 |

## 부록 C. 고전 규칙의 구현

모든 규칙은 알고리즘 1을 공유하고 5단계만 다르며, 각 규칙은 축소 instance ($A'$, $\mathbf{b}'$)와 relaxation vertex $\mathbf{x}^{LP}$를 받는다. most-integral과 most-fractional은 $\arg\max_j |x^{LP}_j - 1/2|$와 $\arg\min_j |x^{LP}_j - 1/2|$로 분기하고 반올림 값을 먼저 시도한다. maxSD: 자유 변수 $r_i$개와 우변 $b'_i$를 가진 0/1 등식 행에서 $x_j=1$의 solution density는 그 행의 모든 변수에 대해 $b'_i/r_i$, $x_j=0$은 $1-b'_i/r_i$다; 규칙은 모든 행과 변수에 걸쳐 밀도가 최대인 변수–값 쌍을 취하고, 동률은 그

변수를 포함하는 행들의 밀도 합으로 깬다. dom/wdeg: 모든 자유 변수의 domain 크기가 2이므로 규칙은 conflict 가중 차수의 최대로 환원된다; 원래 행마다 가중치(초기 0)가 있어 2~3단계의 bound check나 propagation이 그 행 때문에 node를 버릴 때마다 1씩 늘고, 변수의 점수는 그 변수가 나타나는 행들의 (1+가중치) 합이며 반올림 LP 값을 먼저 시도한다. impact: node에서 모든 자유 변수와 두 값에 대해 값을 고정하고 propagation을 돌린다; 쌍의 impact는 1 더하기 propagation이 고정하는 변수 수이고, 모순인 쌍은 반증된 것으로 기록해(LP probing처럼) 반대 값을 고정하며, 규칙은 impact 합이 최대인 변수로 분기하고 impact가 작은 값을 먼저 시도한다. LP probing: 모든 자유 변수와 두 값에 대해 그 값을 고정한 relaxation을 다시 풀고, relaxation이 infeasible한 값은 반증해 반대 값을 고정하며, 반증이 없으면 most-integral 선택으로 돌아간다. uniform: 자유 변수와 값을 균등하게 뽑는다. oracle: 모든 node에서 CP-SAT로 $S(F,\mathbf{v})$를 열거하고 정확한 marginal에 규칙 (5)를 적용한다. BP: 정확한 cardinality-factor message를 prefix–suffix convolution으로 계산하는 damped sum-product(damping 0.9, 허용오차 $10^{-5}$, 최대 300회)이며 추정치에 규칙 (5)를 적용한다.

## 부록 D. 타이밍 복제

표 A3. 평가 seed 셋에 걸친 학습 arm의 타이밍 복제(seed 0이 본문 표에 보고한 것): 해결 instance 수와 wall-clock 중앙값. 탐색은 relaxation vertex가 주어지면 결정적이므로 node 수는 seed 간에 동일하다.

| 크기 | arm | seed 0 | seed 1 | seed 2 |
|---|---|---|---|---|
| 21×60 | Stage 1 | 100/100, 0.65 s | 100/100, 0.48 s | 100/100, 0.49 s |
| 21×60 | Stage 1+2 | 100/100, 0.36 s | 100/100, 0.27 s | 100/100, 0.26 s |
| 24×70 | Stage 1 | 100/100, 6.60 s | 100/100, 6.50 s | 100/100, 6.42 s |
| 24×70 | Stage 1+2 | 100/100, 2.39 s | 100/100, 1.94 s | 100/100, 1.93 s |
| 28×80 | Stage 1 | 90/100, 182.65 s | 90/100, 182.50 s | 90/100, 182.38 s |
| 28×80 | Stage 1+2 | 100/100, 46.61 s | 100/100, 46.30 s | 100/100, 46.34 s |

node 수는 seed 불변이다; 표 A3은 학습 arm의 wall-clock을 평가 seed 두 개로 더 복제한다. 작은 두 크기의 seed 0 run은 더 가벼운 동시 부하에서 돈 seed 1, 2보다 20~30% 느리며, 본문의 비율은 전부 seed 0을 쓰므로 학습 arm에 보수적이다. 28×80에서 세 seed는 1% 안에서 일치한다.

## 데이터와 코드 공개

instance 생성기, 학습·평가 코드, 학습된 network, 모든 표의 instance별 기록은 출판 시 공개 저장소에 공개한다.

## 참고문헌


Aardal, K., Bixby, R. E., Hurkens, C. A. J., Lenstra, A. K., and Smeltink, J. W. (2000). Market split and basis reduction: Towards a solution of the Cornuéjols–Dawande instances. INFORMS Journal on Computing, 12(3):192–202.

Aardal, K., Hurkens, C. A. J., and Lenstra, A. K. (2000). Solving a system of linear Diophantine equations with lower and upper bounds on the variables. Mathematics of Operations Research, 25(3):427–442.

Achterberg, T., Koch, T., and Martin, A. (2005). Branching rules revisited. Operations Research Letters, 33(1):42–54.

Amos, B. (2023). Tutorial on amortized optimization. Foundations and Trends in Machine Learning, 16(5):592–732.

Bestuzheva, K., Besançon, M., Chen, W.-K., et al. (2021). The SCIP Optimization Suite 8.0. Technical Report 21-41, Zuse Institute Berlin.

Boussemart, F., Hemery, F., Lecoutre, C., and Sais, L. (2004). Boosting systematic search by weighting constraints. In ECAI, pages 146–150.

Cappart, Q., Moisan, T., Rousseau, L.-M., Prémont-Schwarz, I., and Cire, A. A. (2021). Combining reinforcement learning and constraint programming for combinatorial optimization. In AAAI Conference on Artificial Intelligence.

Chen, Z., Liu, J., Wang, X., Lu, J., and Yin, W. (2023). On representing mixed-integer linear programs by graph neural networks. In International Conference on Learning Representations.

Chu, G. and Stuckey, P. J. (2015). Learning value heuristics for constraint programming. In CPAIOR.

Cornuéjols, G. and Dawande, M. (1999). A class of hard small 0-1 programs. In Integer Programming and Combinatorial Optimization, pages 284–293.

Etheve, M., Alès, Z., Bissuel, C., Juan, O., and Kedad-Sidhoum, S. (2020). Reinforcement learning for variable selection in a branch and bound algorithm. In CPAIOR, pages 176–185.

Feng, S. and Yang, Y. (2025). SORREL: Suboptimal-demonstration-guided reinforcement learning for learning to branch. In AAAI Conference on Artificial Intelligence.

Gasse, M., Chételat, D., Ferroni, N., Charlin, L., and Lodi, A. (2019). Exact combinatorial optimization with graph convolutional neural networks. In Advances in Neural Information Processing Systems, volume 32.

Gershman, S. J. and Goodman, N. D. (2014). Amortized inference in probabilistic reasoning. In Proceedings of the Annual Meeting of the Cognitive Science Society, volume 36.

Gurobi Optimization, LLC (2024). Gurobi Optimizer Reference Manual.

Han, Q., Yang, L., Chen, Q., Zhou, X., Zhang, D., Wang, A., Sun, R., and Luo, X. (2023). A GNN-guided predict-and-search framework for mixed-integer linear programming. In International Conference on Learning Representations.

Haralick, R. M. and Elliott, G. L. (1980). Increasing tree search efficiency for constraint satisfaction problems. Artificial Intelligence, 14(3):263–313.

Hottung, A., Tanaka, S., and Tierney, K. (2020). Deep learning assisted heuristic tree search for the container pre-marshalling problem. Computers & Operations Research, 113:104781.

Khalil, E. B., Le Bodic, P., Song, L., Nemhauser, G., and Dilkina, B. (2016). Learning to branch in mixed integer programming. In AAAI Conference on Artificial Intelligence.

Khalil, E. B., Morris, C., and Lodi, A. (2022). MIP-GNN: A data-driven framework for guiding combinatorial solvers. In AAAI Conference on Artificial Intelligence.

Lenstra, A. K., Lenstra, H. W., and Lovász, L. (1982). Factoring polynomials with rational coefficients. Mathematische Annalen, 261(4):515–534.

Li, Z. and Si, X. (2022). NSNet: A general neural probabilistic framework for satisfiability problems. In Advances in Neural Information Processing Systems, volume 35.

Li, Z., Guo, J., and Si, X. (2023). G4SATBench: Benchmarking and advancing SAT solving with graph neural networks. Transactions on Machine Learning Research.

Marques-Silva, J. P. and Sakallah, K. A. (1999). GRASP: A search algorithm for propositional satisfiability. IEEE Transactions on Computers, 48(5):506–521.

Mazyavkina, N., Sviridov, S., Ivanov, S., and Burnaev, E. (2021). Reinforcement learning for combinatorial optimization: A survey. Computers & Operations Research, 134:105400.

Montanari, A., Ricci-Tersenghi, F., and Semerjian, G. (2007). Solving constraint satisfaction problems through belief propagation-guided decimation. In Allerton Conference on Communication, Control, and Computing.

Nair, V., Bartunov, S., Gimeno, F., et al. (2020). Solving mixed integer programs using neural networks. arXiv:2012.13349.

Ohrimenko, O., Stuckey, P. J., and Codish, M. (2009). Propagation via lazy clause generation. Constraints, 14(3):357–391.

Parsonson, C. W. F., Laterre, A., and Barrett, T. D. (2023). Reinforcement learning for branch-and-bound optimisation using retrospective trajectories. In AAAI Conference on Artificial Intelligence.

Perron, L. and Didier, F. (2023). OR-Tools CP-SAT. Google.

Pesant, G., Quimper, C.-G., and Zanarini, A. (2012). Counting-based search: Branching heuristics for constraint satisfaction problems. Journal of Artificial Intelligence Research, 43:173–210.

Pryor, J. and Chinneck, J. W. (2011). Faster integer-feasibility in mixed-integer linear programs by branching to force change. Computers & Operations Research, 38(8):1143–1152.

Refalo, P. (2004). Impact-based search strategies for constraint programming. In Principles and Practice of Constraint Programming (CP), pages 557–571.

Scavuzzo, L., Chen, F. Y., Chételat, D., Gasse, M., Lodi, A., Yorke-Smith, N., and Aardal, K. (2022). Learning to branch with tree MDPs. In Advances in Neural Information Processing Systems, volume 35.

Schnorr, C. P. and Euchner, M. (1994). Lattice basis reduction: Improved practical algorithms and solving subset sum problems. Mathematical Programming, 66:181–199.

Selsam, D., Lamm, M., Bünz, B., Liang, P., de Moura, L., and Dill, D. L. (2019). Learning a SAT solver from single-bit supervision. In International Conference on Learning Representations.

Smit, I. G., Zhou, J., Reijnen, R., Wu, Y., Chen, J., Zhang, C., Bukhsh, Z., Zhang, Y., and Nuijten, W. (2025). Graph neural networks for job shop scheduling problems: A survey. Computers & Operations Research, 176:106914.

The FPLLL development team (2023). fpylll, a Python wrapper for fplll. https://github.com/fplll/fpylll.

Vaezipoor, P., Lederman, G., Wu, Y., Maddison, C., Grosse, R. B., Seshia, S. A., and Bacchus, F. (2021). Learning branching heuristics for propositional model counting. In AAAI Conference on Artificial Intelligence.

Velickovic, P., Cucurull, G., Casanova, A., Romero, A., Liò, P., and Bengio, Y. (2018). Graph attention networks. In International Conference on Learning Representations.

Wassermann, A. (2025). Solving the market split problem with lattice enumeration. arXiv:2508.08702; Mathematical Programming Computation, to appear.

Williams, R. J. (1992). Simple statistical gradient-following algorithms for connectionist reinforcement learning. Machine Learning, 8(3–4):229–256.

Yan, F., Clautiaux, F., Froger, A., and Albar, B. (2025). Generic machine-learning-augmented beam search for resource-constrained shortest path reformulations of combinatorial optimization problems. Computers & Operations Research, 187:107339.